

به نام ایزد



گزارش کتبی پروژه کارشناسی

موضوع:

طراحی و ارزیابی ضرب کننده تقریبی مبتنی بر توان های دو برای شتاب دهی
سخت افزاری شبکه های عصبی^۱

استاد: دکتر حاتم عبدلی

تهیه شده توسط:

سارا راشدی: ۴۰۱۱۲۳۵۸۰۱۷

رومینا خان محمدی: ۴۰۱۱۲۳۵۸۰۱۲

مدیسا خبازی: ۴۰۱۱۲۳۵۸۰۱۳

^۱ Design and Evaluation of a Power-of-Two-Based Approximate Multiplier for Hardware Acceleration of Neural Networks

فهرست

چکیده.....	۶
مقدمه.....	۷
۱. فصل اول : کلیات پژوهش.....	۹
۱.۱. پیش‌زمینه و انگیزه‌ی پژوهش.....	۱۰
۱.۲. بیان مسئله.....	۱۰
۱.۳. اهمیت و ضرورت انجام تحقیق.....	۱۱
۱.۴. اهداف پژوهش.....	۱۱
هدف اصلی.....	۱۱
اهداف فرعی.....	۱۱
۱.۵. سهم و نوآوری‌های کار.....	۱۲
۱.۶. دامنه و محدودیت‌های پژوهش.....	۱۳
۲. فصل دوم : مبانی محاسبات تقریبی.....	۱۴
۲.۱. خاستگاه محاسبات تقریبی و بحران مقیاس‌پذیری سخت‌افزار.....	۱۵
۲.۲. انگیزه و چرایی محاسبات تقریبی.....	۱۵
۲.۳. سطوح مختلف اعمال تقریب در سیستم‌های محاسباتی.....	۱۶
۲.۳.۱. سطح نرم‌افزار و الگوریتم.....	۱۷
۲.۳.۲. سطح معماری و سیستم حافظه.....	۱۷
۲.۳.۳. سطح مدار و حساب دیجیتال (موضوع این پژوهش).....	۱۸
۲.۳.۴. سطح ولتاژ و ترانزیستور.....	۱۸

۲.۴. دسته‌بندی خطاهای حسابی و مفاهیم ریاضی	۱۸
۲.۴.۱. خطای مطلق	۱۸
۲.۵. معیارهای آماری سنجش خطای واحدهای حسابی	۱۹
۲.۵.۱. نرخ خطا	۱۹
۲.۵.۲. میانگین خطای قدرمطلق	۱۹
۲.۵.۳. میانگین خطای نسبی	۲۰
۲.۵.۴. خطای نرمال شده	۲۰
۲.۵.۶. میانگین توان دوم خطاها	۲۰
۲.۶. مصالحه میان کارایی سخت‌افزاری و کیفیت محاسبات	۲۰
۲.۷. تحلیل اثر خطاهای حسابی بر استنتاج شبکه‌های عصبی	۲۲
۳. فصل سوم: ضرب‌کننده‌های تقریبی و فشرده‌سازها	۲۴
۳.۱. آناتومی عملیات ضرب در شبکه‌ی عصبی	۲۵
۳.۲. ساختار مفهومی ضرب‌کننده‌ی دقیق	۲۵
۳.۳. محدودیت ضرب عمومی برای هدف این پژوهش	۲۵
۳.۴. مبنای ریاضی ضرب مبتنی بر توان دو	۲۶
۳.۵. نقش Pruning در مسیر PoT/Shift	۲۶
۳.۶. نمایش سخت‌افزاری وزن‌های PoT	۲۸
۳.۷. مقایسه‌ی مفهومی مسیر PoT/Shift و Exact	۲۸
۴. فصل چهارم: هوش مصنوعی، شبکه عصبی و طبقه‌بندی تصویر	۳۰
۴.۱. ضرورت و جایگاه شبکه‌های عصبی در طبقه‌بندی تصویر	۳۱
۴.۲. معرفی دیتاست	۳۱
۴.۳. لایه‌های مدل	۳۲

۴.۳.۱	لایه ورودی	۳۳
۴.۳.۲	لایه مسطح‌سازی	۳۳
۴.۳.۳	لایه تمام‌متصل اول	۳۳
۴.۳.۴	لایه حذف کردن	۳۴
۴.۳.۵	لایه تمام‌متصل دوم	۳۴
۴.۴	اعمال توان دوو هرس کردن	۳۶
۴.۵	اثر خطای ضرب در بخش های محاسباتی و تاب‌آوری خطا	۳۸
۵	فصل پنجم: روش پیشنهادی و پیاده‌سازی سخت‌افزاری در hls۴ml	۳۹
۵.۱	متدولوژی کلی و جریان کار	۴۰
۵.۱.۱	گام اول: طراحی و آموزش مدل پایه در Keras/TensorFlow	۴۱
۵.۱.۲	گام دوم: تبدیل مدل و نگاشت به نمایش توان دو (PoT) با hls۴ml	۴۱
۵.۱.۳	گام سوم: بازطراحی هدرهای حسابی و تزریق ضرب‌کننده سفارشی	۴۱
۵.۱.۴	گام چهارم: سنتز، ارزیابی منابع و شبیه‌سازی در Vitis HLS	۴۱
۵.۲	چارچوب hls4ml	۴۲
۵.۳	مدل‌سازی عددی و کوانتیزه‌سازی به توان دو (PoT)	۴۳
۵.۴	تزریق ضرب‌کننده سفارشی PoT به hls۴ml	۴۴
۵.۵	استراتژی‌های بهینه‌سازی	۴۵
۵.۶	تفکیک شبیه‌سازی نرم‌افزاری (C.Sim) و سنتز سخت‌افزاری (C.Synthesis)	۴۵
۶	فصل ششم: آزمایش‌ها، نتایج و تحلیل	۴۷
۶.۱	چارچوب آزمون تجربی و تنظیمات پیاده‌سازی	۴۸
۶.۲	سنجش عملکرد الگوریتمی	۴۸
۶.۳	سیر تکامل سلسله‌مراتبی و فازهای بهینه‌سازی سخت‌افزاری	۴۹

۵۰	۶.۴. ارزیابی جامع نتایج سنتز و مصرف منابع فیزیکی
۵۳	۶.۵. تحلیل جامع مهندسی و بازخوانی دستاوردهای سخت‌افزاری
۵۴	۶.۶. تحلیل رفتار توان مصرفی، پایداری حرارتی و بهره‌وری انرژی
۵۶	۷. فصل هفتم: جمع‌بندی و کارهای آینده
۵۷	۷.۱. جمع‌بندی یافته‌ها
۵۷	۷.۲. ارزیابی تطبیقی در بستر آکادمیک
۵۷	۷.۳. محدودیت‌های ذاتی پژوهش
۵۸	۷.۴. پیشنهادات برای پژوهش‌های آتی
۶۰	منابع و پیوست

چکیده

با گسترش کاربرد شبکه‌های عصبی عمیق^۲ در سامانه‌های پردازش تصویر، هزینه محاسباتی عملیات ضرب و جمع به یکی از چالش‌های اصلی در پیاده‌سازی سخت‌افزاری این شبکه‌ها تبدیل شده است. از آن‌جا که بسیاری از کاربردهای پردازش تصویر و یادگیری ماشین^۳ تا حدی نسبت به خطاهای عددی کوچک مقاوم هستند، محاسبات تقریبی^۴ به‌عنوان رویکردی برای ایجاد توازن میان دقت محاسبات و هزینه‌های سخت‌افزاری مطرح شده است.

ایده اصلی این پروژه جایگزین کردن شیوه اجرای برخی ضرب‌ها با ضرب مبتنی بر توان دو^۵ است. در این روش، وزن پس از اعمال هرس کردن^۶، به نزدیک‌ترین مقدار از فرم $2^K \pm$ نگاشت می‌شود. در نتیجه ضرب یک فعال‌سازی در وزن به جای استفاده از ضرب‌کننده عمومی، با یک شیفت^۷ و اعمال علامت انجام می‌شود. هدف از این جایگزینی، کاهش استفاده از منابع اختصاصی FPGA و بهبود ویژگی‌های توان و زمان‌بندی است؛ هرچند انتظار می‌رود بخشی از هزینه سخت‌افزاری به منطق قابل‌برنامه‌ریزی مانند LUT و FF منتقل شود.

برای ارزیابی اثر تقریب در یک کاربرد واقعی، از یک شبکه عصبی کانولوشنی جهت طبقه‌بندی تصاویر^۸ مجموعه‌داده MNIST استفاده شده است. عملکرد مدل در دو حالت ضرب دقیق و ضرب تقریبی بررسی می‌شود و معیارهایی مانند دقت طبقه‌بندی، خطای حاصل‌ضرب و در صورت امکان منابع سخت‌افزاری و تأخیر^۹ پیاده‌سازی تحلیل خواهند شد. هدف نهایی، بررسی میزان تأثیر خطای ناشی از ضرب تقریبی بر خروجی شبکه و ارزیابی امکان استفاده از چنین ساختاری در کاربردهای هوش مصنوعی است.

^۲ Deep Neural Networks

^۳ Machine Learning

^۴ Approximate Computing

^۵ Power-of-Two (POT)

^۶ Pruning

^۷ Shift

^۸ Image Classification

^۹ Latency

مقدمه

در سال‌های اخیر، پیشرفت‌های حاصل‌شده در حوزه هوش مصنوعی^{۱۰} و یادگیری عمیق^{۱۱} باعث شده است که شبکه‌های عصبی در بسیاری از کاربردها، از جمله پردازش تصویر^{۱۲}، تشخیص اشیاء، خودروهای هوشمند، سامانه‌های پزشکی و تجهیزات قابل‌حمل مورد استفاده قرار گیرند. در میان مدل‌های مختلف یادگیری عمیق، شبکه‌های عصبی کانولوشنی نقش مهمی در پردازش و طبقه‌بندی تصاویر دارند. این شبکه‌ها با استخراج تدریجی ویژگی‌های مختلف از تصویر، می‌توانند الگوهای پیچیده را شناسایی کرده و تصاویر را در کلاس‌های از پیش تعیین‌شده قرار دهند.

با گسترش روزافزون کاربردهای هوش مصنوعی، شبکه‌های عصبی به یکی از اجزای اصلی بسیاری از سامانه‌های پردازشی تبدیل شده‌اند. با این حال، اجرای این شبکه‌ها، به‌ویژه در سامانه‌های سخت‌افزاری مبتنی بر FPGA، به دلیل تعداد زیاد عملیات ضرب و جمع، می‌تواند به مصرف قابل‌توجه منابع سخت‌افزاری، توان و زمان پردازش منجر شود. در میان این عملیات، ضرب یکی از پرهزینه‌ترین بخش‌های مسیر محاسباتی شبکه‌های عصبی محسوب می‌شود؛ زیرا پیاده‌سازی ضرب عمومی به منابع محاسباتی اختصاصی مانند DSP نیاز دارد.

یکی از راهکارهای کاهش این هزینه، تغییر نحوه نمایش و پردازش وزن‌های شبکه عصبی است. در این پژوهش، به‌جای استفاده مستقیم از مقادیر عمومی وزن‌ها، از نمایش توان دو یا POT استفاده می‌شود. در این روش، وزن‌های غیرصفر به نزدیک‌ترین توان دو نگاشت می‌شوند و در نتیجه، عملیات ضرب میان ورودی و وزن می‌تواند از یک ضرب عمومی به یک عملیات شیفت تبدیل شود. به این ترتیب، مسیر محاسباتی از نظر استفاده از منابع سخت‌افزاری تغییر می‌کند و امکان کاهش وابستگی به واحدهای DSP فراهم می‌شود.

پژوهش حاضر این ایده را به‌صورت یک مسیر مرحله‌ای بررسی می‌کند و تنها به یک پیاده‌سازی شیفت محدود نمی‌شود. ابتدا یک معماری دقیق^{۱۳} به‌عنوان خط پایه در نظر گرفته می‌شود تا دقت شبکه، منابع سخت‌افزاری، زمان‌بندی، تأخیر و توان مصرفی به‌عنوان معیار مرجع مشخص شوند. سپس در مرحله بعد، مسیر شیفت با توان دو بررسی می‌شود تا اثر جایگزینی ضرب عمومی با عملیات شیفت بر منابع سخت‌افزاری، به‌ویژه DSP، LUT و

^{۱۰} Artificial Intelligence (AI)

^{۱۱} Deep Learning

^{۱۲} Image Processing

^{۱۳} Exact

FF، مشخص شود. در مرحله نهایی نیز Pruning به این مسیر اضافه می‌شود تا علاوه بر ساده‌سازی محاسبات ضرب، وزن‌های کم‌اهمیت نیز از مسیر محاسباتی حذف شوند.

هر مرحله با هدف بررسی یک جنبه مشخص از مصالحه میان دقت و هزینه سخت‌افزاری انجام می‌شود. مسیر Exact نقش مرجع را دارد، مسیر شیفت با توان دو با هدف کاهش استفاده از منابع اختصاصی ضرب و جایگزینی آن با منطق شیفت ارزیابی می‌شود، و مرحله شیفت با توان دو همراه با هرس کردن با هدف کاهش بیشتر تعداد ضرایب فعال و ساده‌سازی مسیر ذخیره‌سازی و محاسبات توسعه می‌یابد.

برای ارزیابی این روش، یک شبکه عصبی سبک برای طبقه‌بندی تصاویر مجموعه‌داده MNIST انتخاب شده و پیاده‌سازی سخت‌افزاری آن با استفاده از Vitis HLS و هدف‌گذاری روی FPGA انجام می‌شود. در این ارزیابی، علاوه بر دقت طبقه‌بندی، معیارهایی مانند LUT، FF، BRAM، DSP، زمان بندی تخمینی^{۱۴}، تاخیر^{۱۵} و توان مصرفی نیز بررسی می‌شوند. هدف نهایی پژوهش، یافتن یک نقطه تعادل مناسب میان کیفیت محاسبات شبکه و هزینه پیاده‌سازی سخت‌افزاری است؛ به گونه‌ای که کاهش منابع اختصاصی و توان مصرفی بدون افت نامطلوب در عملکرد شبکه حاصل شود.

^{۱۴} Estimated Timing

^{۱۵} Latency

۱. فصل اول : کلیات پژوهش

۱.۱. پیش‌زمینه و انگیزه‌ی پژوهش

گسترش مدل‌های یادگیری عمیق و شبکه‌های عصبی کانولوشنی (CNN) در پردازش لبه^{۱۶}، طراحان سخت‌افزار را با چالش جدی در رابطه با محدودیت منابع و توان مصرفی مواجه کرده است. لایه‌های کانولوشن و تمام‌متصل^{۱۷} به شدت متکی بر عملیات ضرب. جمع^{۱۸} هستند؛ عملیاتی که در پیاده‌سازی‌های FPGA، بخش عمده‌ای از بلوک‌های منطقی (LUT) و منابع اختصاصی DSP را می‌بلعد.

انگیزه‌ی اصلی این پژوهش، بهره‌گیری از خاصیت «تحمل ذاتی خطا» در شبکه‌های عصبی برای حل مشکل مصرف منابع است. با جایگزینی ضرب‌کننده‌های دقیق سنتی با واحدهای محاسبات تقریبی در سطح معماری حسابی، می‌توان بدون افت چشمگیر در دقت استنتاج مدل، پیچیدگی مدار و مصرف توان را کاهش داد.

۱.۲. بیان مسئله

در طراحی شتاب‌دهنده‌های شبکه عصبی، تنها حفظ دقت مدل کافی نیست؛ بلکه باید مصرف منابع FPGA، توان، زمان‌بندی و تأخیر نیز در نظر گرفته شوند. ضرب دقیق نتیجه عددی مطلوب را ایجاد می‌کند، اما استفاده گسترده از ضرب‌کننده‌ها می‌تواند مصرف DSP و BRAM را افزایش دهد.

از طرف دیگر، حذف کامل ضرب‌کننده‌های اختصاصی بدون طراحی جایگزین مناسب ممکن است باعث افزایش LUT و FF شود. بنابراین مسئله این پژوهش بررسی یک مصالحه مشخص است: آیا می‌توان با نداشت وزن‌ها به توان‌های دو و اجرای ضرب به صورت شیفت، منابع اختصاصی مانند DSP و BRAM را کاهش داد و در عین حال دقت شبکه و زمان اجرای آن در محدوده قابل قبول باقی بماند.

^{۱۶} Edge Computing

^{۱۷} Fully Connected

^{۱۸} MAC

۱.۳. اهمیت و ضرورت انجام تحقیق

در سامانه‌های نهفته^{۱۹} و تجهیزات پردازش لبه، منابع سخت‌افزاری، بودجه‌ی توان و خنک‌سازی مدار با محدودیت‌های شدیدی مواجه‌اند. اجرای مستقیم مدل‌های سنگین هوش مصنوعی بر روی این تجهیزات بدون بهینه‌سازی در سطح معماری سخت‌افزار عملاً ناممکن است.

ضرورت انجام این پژوهش از آن‌جا ناشی می‌شود که راهکارهای مرسوم نرم‌افزاری (مانند هرس وزن‌ها^{۲۰} یا کوانتیزاسیون^{۲۱}) به تنهایی برای رسیدن به بالاترین سطح بهره‌وری کافی نیستند. تلفیق روش‌های کاهش دقت عددی با طراحی سفارشی واحدهای حسابی تقریبی در سطح معماری سخت‌افزار می‌تواند یک گام کلیدی در جهت طراحی شتاب‌دهنده‌های کم‌مصرف برای هوش مصنوعی لبه باشد.

۱.۴. اهداف پژوهش

هدف اصلی

هدف اصلی، پیاده‌سازی و مقایسه دو مسیر محاسباتی Exact و شیفت با توان دو در یک شبکه عصبی سبک برای طبقه‌بندی MNIST و ارزیابی هم‌زمان کیفیت مدل و شاخص‌های سخت‌افزاری است.

اهداف فرعی

۱. ساخت یک شبکه عصبی سبک و قابل پیاده‌سازی روی FPGA برای MNIST.
۲. اعمال Pruning و سپس نگاشت وزن‌های غیرصفر به نزدیک‌ترین توان دو.
۳. جایگزینی ضرب عمومی در مسیر Dense با عملگر شیفت همراه با اعمال علامت.
۴. پیاده‌سازی هر دو مسیر در Vitis HLS با قالب عددی ثابت و شرایط زمان‌بندی یکسان.
۵. مقایسه منابع FPGA، Latency، Timing Estimated و توان مصرفی دو طراحی.
۶. بررسی افت دقت ناشی از تقریب وزن‌ها و اثر آن بر صحت طبقه‌بندی.

^{۱۹} Embedded Systems

^{۲۰} Weight Pruning

^{۲۱} Quantization

۱.۵. سهم و نوآوری های کار

سهم اصلی این پژوهش، بررسی یک مسیر سخت افزاری برای کاهش هزینه محاسبات ضرب در شبکه عصبی با استفاده از نمایش وزن ها به صورت توان دو یا و تبدیل عملیات ضرب عمومی به عملیات شیفت است. ایده اصلی این است که به جای پیاده سازی ضرب کننده های عمومی برای تمام وزن ها، وزن های شبکه پس از آموزش به شکلی نمایش داده شوند که هر وزن غیر صفر را بتوان با یک علامت و یک توان صحیح از عدد ۲ بیان کرد. در این حالت، حاصل ضرب یک ورودی در وزن به جای استفاده از ضرب سخت افزاری، با شیفت دادن ورودی و در صورت نیاز اعمال علامت محاسبه می شود.

نوآوری پژوهش تنها در جایگزینی ضرب با شیفت خلاصه نمی شود، بلکه یک زنجیره چند مرحله ای برای بررسی مصالحه میان دقت شبکه، مصرف منابع سخت افزاری، زمان بندی و توان در نظر گرفته شده است. در این مسیر، چند پیکربندی محاسباتی از حالت مرجع Exact تا حالت نهایی مبتنی بر توان دو، هرس کردن و شیفت مورد بررسی قرار می گیرد تا مشخص شود هر مرحله چه اثری بر شاخص های مختلف سیستم دارد.

در پیکربندی اول، معماری Exact به عنوان خط پایه در نظر گرفته می شود. در این حالت وزن ها بدون تقریب مورد استفاده قرار می گیرند و عملیات ضرب در لایه های Dense به صورت ضرب معمولی انجام می شود. این پیکربندی معیار اصلی برای مقایسه دقت، منابع FPGA، زمان بندی، Latency و توان مصرفی است و امکان می دهد اثر هر تغییر سخت افزاری نسبت به یک مرجع مشخص اندازه گیری شود.

در پیکربندی دوم، مسیر شیفت با توان دو بررسی می شود. در این مرحله وزن های غیر صفر به نزدیک ترین توان دو نگاشت می شوند و عملیات ضرب بر اساس رابطه ی

$$x \times w \rightarrow \text{sign}(w) \times (x \ll k)$$

انجام می شود. هدف اصلی این مرحله حذف نیاز به DSP به عنوان واحد ضرب عمومی و انتقال عملیات ضرب به منطق قابل پیکربندی FPGA است. بنابراین انتظار می رود بیشترین اثر این مرحله در کاهش استفاده از DSP و بهبود برخی شاخص های زمان بندی مشاهده شود؛ در مقابل، بخشی از هزینه محاسبات از DSP به LUT و FF منتقل می شود.

در پیکربندی سوم، مسیر ترکیبی Pruning + PoT + Shift مورد استفاده قرار می گیرد. در این مرحله ابتدا وزن های کم اهمیت با استفاده از آستانه مشخص حذف می شوند و سپس وزن های باقی مانده به نزدیک ترین توان

دو نداشت می‌شوند. در نتیجه، علاوه بر حذف ضرب عمومی، تعداد ضرایب غیرصفر نیز کاهش پیدا می‌کند. در پروژه حاضر آستانه Pruning برابر با ($T=0.1$) انتخاب شده است. این مرحله با هدف کاهش هم‌زمان پیچیدگی ذخیره‌سازی وزن‌ها و هزینه محاسباتی مسیر سخت‌افزار انجام شده است.

این ساختار باعث می‌شود نوآوری پژوهش به صورت یک مسئله‌ی چندمعیاره بررسی شود و تنها یک معیار، مانند تعداد DSP، ملاک موفقیت قرار نگیرد. معیارهای ارزیابی شامل دقت طبقه‌بندی، LUT، FF، BRAM، DSP، Latency، Estimated Timing و توان مصرفی هستند. در واقع هدف، پیدا کردن نقطه‌ای مناسب در مصالحه میان کیفیت محاسبات شبکه و هزینه پیاده‌سازی سخت‌افزاری است.

۱.۶. دامنه و محدودیت‌های پژوهش

- پلتفرم سخت‌افزاری: قطعه FPGA هدف مورد نظر سری Xilinx Alveo U250 بوده و توصیف رفتاری بر پایه محیط‌های HLS توسعه یافته است.
- این پژوهش بر شبکه‌ای سبک از نوع کاملاً متصل^{۲۲} برای MNIST متمرکز است. بنابراین موضوع کار طراحی یک ضرب‌کننده تقریبی سنتی، فشرده‌ساز^{۲۳} ۴ به ۲ یا ساختارهای کلاسیک کاهش حاصل‌ضرب‌های جزئی نیست. تقریب اصلی در سطح وزن‌های شبکه و سپس در سطح معماری اجرای ضرب اعمال می‌شود؛ به این صورت که وزن‌های باقی‌مانده پس از Pruning به فرم توان دو نزدیک شده و ضرب آن‌ها به شیفت تبدیل می‌شود.
- محدودیت شبیه‌سازی: به دلیل ماهیت تفکیک آرایه‌های بیتی و باز شدن کامل حلقه‌ها در ضرب‌کننده سفارشی C++، شبیه‌سازی نرم‌افزاری Csimulation در HLS زمان‌بر بوده و ارزیابی استنتاج در ابعاد بزرگ نیازمند مدیریت منابع محاسباتی است.

^{۲۲} Fully Connected

^{۲۳} Compressor

۲. فصل دوم : مبانی محاسبات تقریبی

۲.۱. خاستگاه محاسبات تقریبی و بحران مقیاس‌پذیری سخت‌افزار

در دهه‌های اخیر، قانون مور و قانون مقیاس‌پذیری ندارد با موانع فیزیکی جدی نظیر پدیده سیلیکون تاریک^{۲۴} و محدودیت‌های اتلاف توان حرارتی مواجه شده‌اند. از سوی دیگر رشد انفجاری حجم داده‌ها و الگوریتم‌های هوش مصنوعی نیازمند توان پردازشی فراتر از قابلیت معماری‌های سنتی است. در این میان محاسبات تقریبی به عنوان یک راهکار اثرگذار مطرح شده است که با بازتعریف مرزهای دقت محاسباتی، کارایی انرژی و سرعت پردازش را به میزان چشمگیری ارتقا می‌دهد. در این فصل، اصول بنیادین محاسبات تقریبی، دلایل کارآمدی آن در پردازش سیگنال و هوش مصنوعی، سطوح مختلف اعمال تقریب و در نهایت معیارهای استاندارد ریاضی جهت سنجش خطا بررسی می‌شوند.

۲.۲. انگیزه و چرایی محاسبات تقریبی

به‌طور سنتی، سیستم‌های دیجیتال بر پایه تضمین صحت ۱۰۰ درصدی نتایج ریاضی^{۲۵} طراحی می‌شوند. این قید سخت‌گیرانه مستلزم تخصیص مساحت سیلیکونی قابل توجه، تأخیر زمانی بالا به دلیل زنجیره‌های طولانی انتشار رقم نقلی^{۲۶} و در نتیجه مصرف توان بالا است. با این حال، دسته‌ای گسترده از کاربردهای امروزی دارای تحمل ذاتی خطا هستند. دلایل این تحمل‌پذیری عبارتند از:

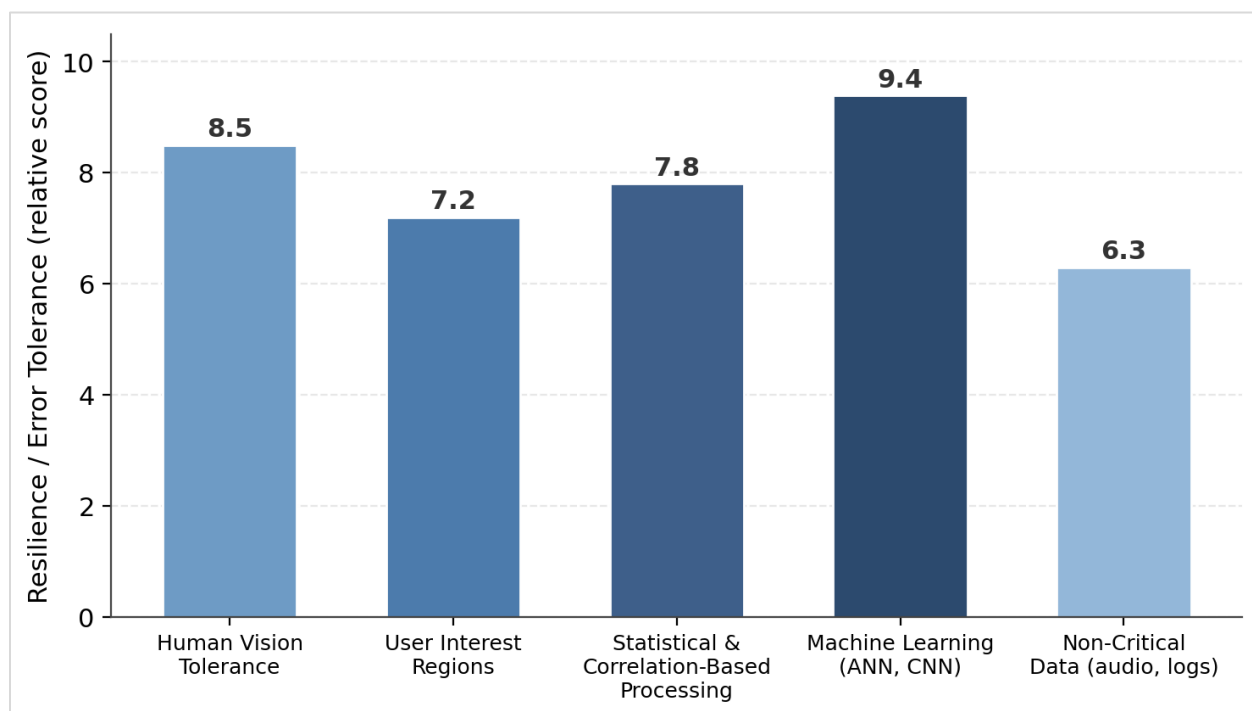
- طبیعت داده‌های ورودی: داده‌های حسی دنیای واقعی (مانند تصاویر، صدا و داده‌های حسگرها) ذاتاً نویزی و غیرقطعی هستند؛ بنابراین دقت بی‌نهایت در محاسبات، اطلاعات بیشتری نسبت به کیفیت ذاتی ورودی تولید نمی‌کند.
- محدودیت ادراک انسان: در کاربردهای چندرسانه‌ای و پردازش تصویر، سیستم بینایی انسان (HVS) قادر به تشخیص خطاهای بسیار کوچک در مقادیر پیکسل‌ها یا ویژگی‌های سطح پایین نیست.
- طبیعت آماری الگوریتم‌های یادگیری ماشین: شبکه‌های عصبی عمیق، ساختارهایی مبتنی بر احتمالات و برازش آماری هستند. این الگوریتم‌ها برای دستیابی به همگرایی نهایی، نیازی به دقت مطلق در تک‌تک ضرب‌ها و جمع‌های میانی ندارند.

^{۲۴} Dark Silicon

^{۲۵} Exactness

^{۲۶} Carry Propagation Chain

بنابراین، با اعمال تقریب هدفمند، می‌توان منابع سخت‌افزاری را برای محاسباتی که تاثیری در تصمیم نهایی ندارند آزاد کرد. تعامل این سه عامل کلیدی در تصویر ۱-۲ خلاصه شده است.



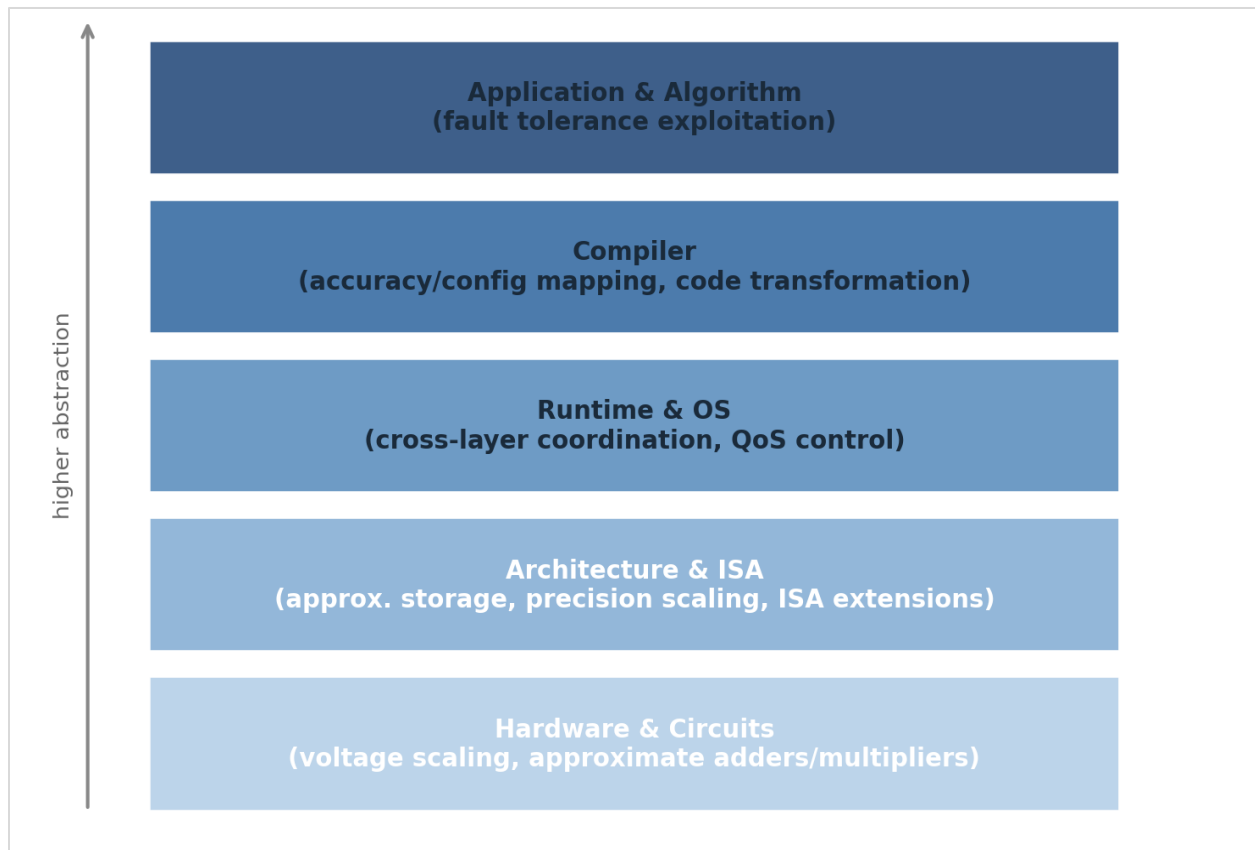
تصویر ۱-۲: پیشران‌ها و عوامل بنیادین تحمل‌پذیری ذاتی خطا در کاربردهای نوین پردازشی و هوش مصنوعی.

۲.۳. سطوح مختلف اعمال تقریب در سیستم‌های محاسباتی

محاسبات تقریبی یک تکنیک تک‌بعدی نیست، بلکه در سراسر پشته محاسباتی^{۲۷} از سطح نرم‌افزار تا فیزیک سیلیکون قابل اعمال است.

همان‌طور که در تصویر ۲-۲ نشان داده شده است، لایه‌های مختلف پشته محاسباتی سطوح متفاوتی از انتزاع و پتانسیل صرفه‌جویی را ارائه می‌دهند که در این میان، لایه مدار و حساب دیجیتال تمرکز اصلی این پژوهش است.

^{۲۷} Computing Stack



تصویر ۲-۲: پشته لایه‌ای اعمال محاسبات تقریبی و موقعیت بهینه‌سازی‌های مداری/حسابی در میان سایر لایه‌ها.

۲.۳.۱. سطح نرم‌افزار و الگوریتم

در این سطح، تقریب از طریق اصلاح الگوریتم‌ها بدون تغییر در سخت‌افزار پایه صورت می‌پذیرد. تکنیک‌هایی مانند نادیده گرفتن برخی تکرارها در حلقه‌ها^{۲۸}، هرس وزن‌های کوچک در شبکه‌های عصبی و کوانتیزاسیون داده‌ها از اعشاری ۳۲ بیتی به قالب‌های نقطه ثابت ۸ یا ۱۶ بیتی در این رده قرار می‌گیرند.

۲.۳.۲. سطح معماری و سیستم حافظه

در این سطح، ذخیره‌سازی داده‌ها به صورت تقریبی انجام می‌شود؛ به عنوان مثال با کاهش زمان رفرش در حافظه‌های DRAM جهت کاهش توان، یا نادیده گرفتن بیت‌های کم‌ارزش داده‌ها در خطوط حافظه نهان^{۲۹} و گذرگاه‌های داده.

^{۲۸} Loop Perforation

^{۲۹} Cache

۲.۳.۳. سطح مدار و حساب دیجیتال (موضوع این پژوهش)

در این لایه، ساختار مدارهای منطقی و ریاضیاتی بازطراحی می‌شوند. هدف اصلی، کاهش تعداد گیت‌های منطقی، حذف زنجیره‌های طولانی انتشار رقم نقلی و کاهش ضریب فعالیت سوئیچینگ ترانزیستورها است. طراحی جمع‌کننده‌ها و ضرب‌کننده‌های تقریبی و فشرده‌سازهای بیتی در این لایه قرار دارند که مستقیماً باعث صرفه‌جویی در مساحت و توان تراشه^{۳۰} یا منابع منطقی LUT/DSP در FPGA می‌شوند.

۲.۳.۴. سطح ولتاژ و ترانزیستور

شامل اعمال تکنیک مقیاس‌بندی ولتاژ بیش از حد مجاز^{۳۱} است؛ به این صورت که ولتاژ تغذیه مدار به زیر حد آستانه کاهش می‌یابد تا توان مصرفی مدار به‌صورت توان دوم کاهش یابد حتی اگر منجر به بروز خطاهای زمان‌بندی^{۳۲} در مسیرهای بحرانی شود.

۲.۴. دسته‌بندی خطاهای حسابی و مفاهیم ریاضی

برای ارزیابی رفتار یک واحد حسابی تقریبی، ابتدا باید خروجی آن به زبان ریاضی مدل‌سازی شود. فرض کنید X و Y دو عدد ورودی باشند. حاصل عملیات دقیق به‌صورت $S(X, Y)$ و حاصل عملیات تقریبی به‌صورت $\tilde{S}(X, Y)$ تعریف می‌شود.

۲.۴.۱. خطای مطلق^{۳۳}

فاصله خطا یا اندازه مطلق خطا، تفاضل مستقیم مقدار دقیق و مقدار تقریبی است:

$$ED(X, Y) = | \tilde{S}(X, Y) - S(X, Y) |$$

۲.۴.۲. خطای نسبی^{۳۴}

^{۳۰} ASIC

^{۳۱} Voltage Overscaling

^{۳۲} Timing Violations

^{۳۳} Error Distance – ED

^{۳۴} Relative Error Distance – RED

نسبت خطای مطلق به مقدار خروجی دقیق را خطای نسبی می‌نامند (به شرطی که خروجی دقیق مخالف صفر باشد):

$$RED(X, Y) = \frac{|\tilde{S}(X, Y) - S(X, Y)|}{|S(X, Y)|} = \frac{ED(X, Y)}{|S(X, Y)|}, S(X, Y) \neq 0$$

۲.۵. معیارهای آماری سنجش خطای واحدهای حسابی^{۳۵}

۲.۵.۱. نرخ خطا^{۳۶}

نرخ خطا، احتمال رخداد هرگونه خطا (صرف‌نظر از بزرگی آن) در میان کل حالات ورودی ممکن است:

$$ER = \frac{\sum_{i=1}^N \mathbb{I}(ED_i \neq 0)}{N}$$

که در آن $\mathbb{I}(\cdot)$ تابع نشان‌گر است که در صورت برقرار بودن شرط داخل، مقدار ۱ و در غیر این صورت مقدار ۰ دارد.

۲.۵.۲. میانگین خطای قدرمطلق^{۳۷}

این معیار که به میانگین فاصله خطا^{۳۸} نیز مشهور است، بزرگی متوسط خطاها را نشان می‌دهد و معیاری کلیدی برای سنجش دقت مدارهای حسابی است:

$$MAE = MED = \frac{1}{N} \sum_{i=1}^N ED_i = \frac{1}{N} \sum_{i=1}^N |\tilde{S}_i - S_i|$$

^{۳۵} Error Metrics

^{۳۶} Error Rate – ER

^{۳۷} Mean Absolute Error – MAE

^{۳۸} Mean Error Distance – MED

۲.۵.۳. میانگین خطای نسبی^{۳۹}

از آنجا که اثر یک مقدار خطای ثابت در ورودی‌های کوچک بسیار مخرب‌تر از ورودی‌های بزرگ است، این معیار میانگین خطای نسبی را محاسبه می‌کند:

$$MRED = \frac{1}{N} \sum_{i=1}^N R ED_i = \frac{1}{N} \sum_{i=1}^N \frac{|\tilde{S}_i - S_i|}{|S_i|}, S_i \neq 0$$

۲.۵.۴. خطای نرمال‌شده^{۴۰}

برای مقایسه منصفانه دو ضرب‌کننده با پهنای بیتی متفاوت (مثلاً ۸ بیتی در برابر ۳۲ بیتی)، میانگین فاصله خطا بر حداکثر مقدار ممکن خروجی (S_{max}) تقسیم می‌شود:

$$NMED = \frac{MED}{S_{max}}$$

۲.۵.۶. میانگین توان دوم خطاها^{۴۱}

این معیار وزن بیشتری به خطاهای بزرگ می‌دهد و در کاربردهای پردازش سیگنال و بینایی ماشین بسیار حائز اهمیت است:

$$MSE = \frac{1}{N} \sum_{i=1}^N (\tilde{S}_i - S_i)^2$$

۲.۶. مصالحه میان کارایی سخت‌افزاری و کیفیت محاسبات^{۴۲}

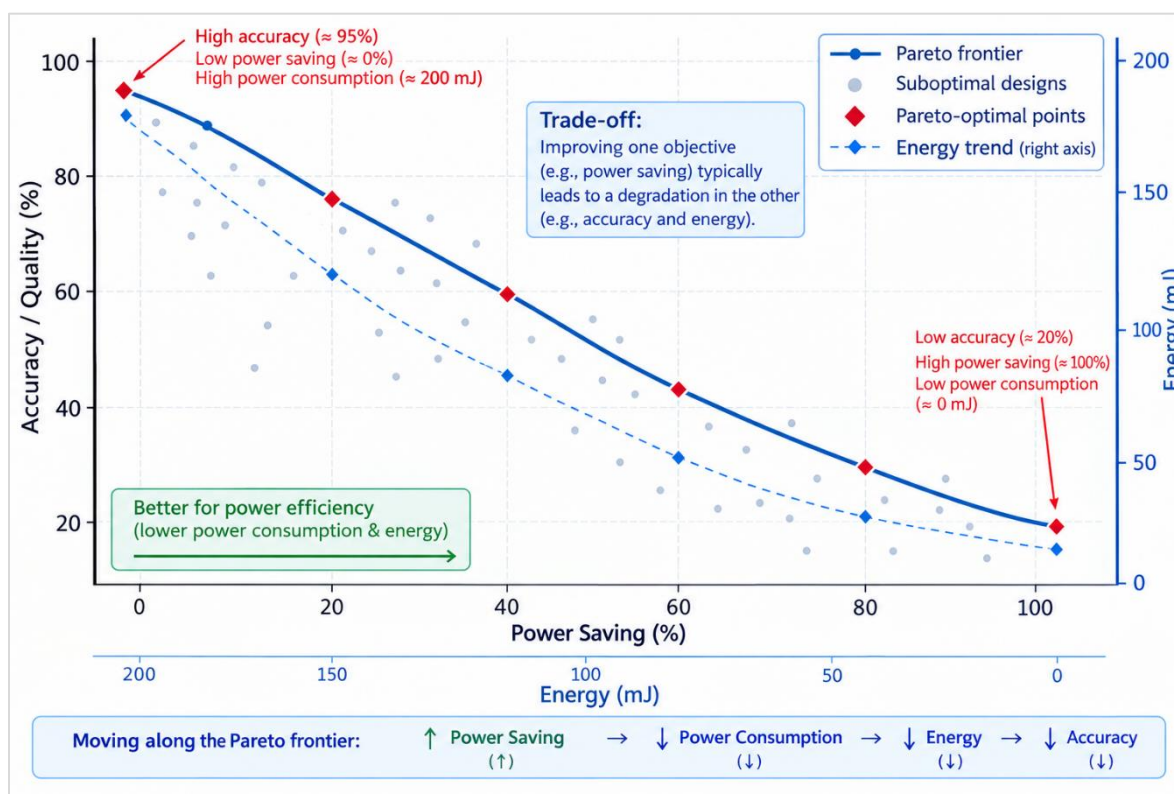
^{۳۹} Mean Relative Error Distance – MRED

^{۴۰} Normalized Mean Error Distance – NMED

^{۴۱} Mean Squared Error – MSE

^{۴۲} Trade-off Analysis

هدف نهایی در محاسبات تقریبی، دستیابی به بهینگی پارتو^{۴۳} میان معیارهای کیفیت و معیارهای هزینه ساخت افزاری است. شکل ۲-۳ نمونه‌ای از این فضای طراحی و مرز مصالحه بهینه را به تصویر می‌کشد؛ طرح‌های قرارگرفته بر روی منحنی پارتو، به ازای هر سطح مشخص از افت دقت، بیشترین میزان بازدهی توان و مساحت را فراهم می‌آورند.



شکل ۲-۳: منحنی جبهه بهینگی پارتو در مصالحه میان دقت مدل و صرفه‌جویی ساخت افزاری (توان/مساحت)

معیارهای کلیدی در این مصالحه به دو دسته تقسیم می‌شوند:

- معیارهای ساخت افزاری:

- مساحت / منابع منطقی: تعداد جدول‌های جستجو (LUTs)، فلیپ‌فلاپ‌ها (FFs) و بلوک‌های DSP در FPGA.

^{۴۳} Pareto Optimality

- تأخیر زمانی : طولانی‌ترین مسیر انتشار سیگنال که بیشینه فرکانس کاری (F_{max}) را تعیین می‌کند.
- توان مصرفی^{۴۴}: شامل توان پویا (وابسته به نرخ کلیدزنی) و توان ایستا (نشتی).
- حاصل ضرب توان در تأخیر^{۴۵}: معیار سنجش بازدهی انرژی به ازای هر عملیات.
- معیارهای کیفیت کاربرد^{۴۶}:
- در پردازش تصویر: نسبت سیگنال به نویز بیشینه (PSNR) و شباهت ساختاری (SSIM).
- در شبکه‌های عصبی (موضوع این پروژه): دقت طبقه‌بندی تصویر^{۴۷} و خطای از دست رفته^{۴۸}.

۲.۷. تحلیل اثر خطاهای حسابی بر استنتاج شبکه‌های عصبی

معیارهای آماری معرفی شده صرفاً رفتارهای محلی یک ضرب‌کننده را توصیف می‌کنند؛ اما در شبکه‌های عصبی، ضرب‌کننده‌ها در زنجیره‌ای از لایه‌ها (بیشینه‌گیری و توابع فعال‌سازی) قرار می‌گیرند. خطاهای دارای بایاس غیرصفر (میانگین خطای مثبت یا منفی مداوم) می‌توانند در طول لایه‌های شبکه انباشته شده و توزیع ویژگی‌ها را از تعادل خارج کنند. در مقابل، تقریب‌هایی که نرخ خطای پایین دارند و خطای آن‌ها تنها در حالت‌های ورودی نادر رخ می‌دهد (مانند تقریب در حالت تمام‌یک)، کمترین انحراف را در بردار ویژگی لایه‌های پایانی ایجاد می‌کنند. نحوه انتشار و انباشتگی این دو نوع خطا در طول لایه‌های متوالی شبکه عصبی در شکل ۲-۴ مقایسه شده است. این اصل، مبنای انتخاب و طراحی فشرده‌ساز تقریبی در فصل بعد خواهد بود.

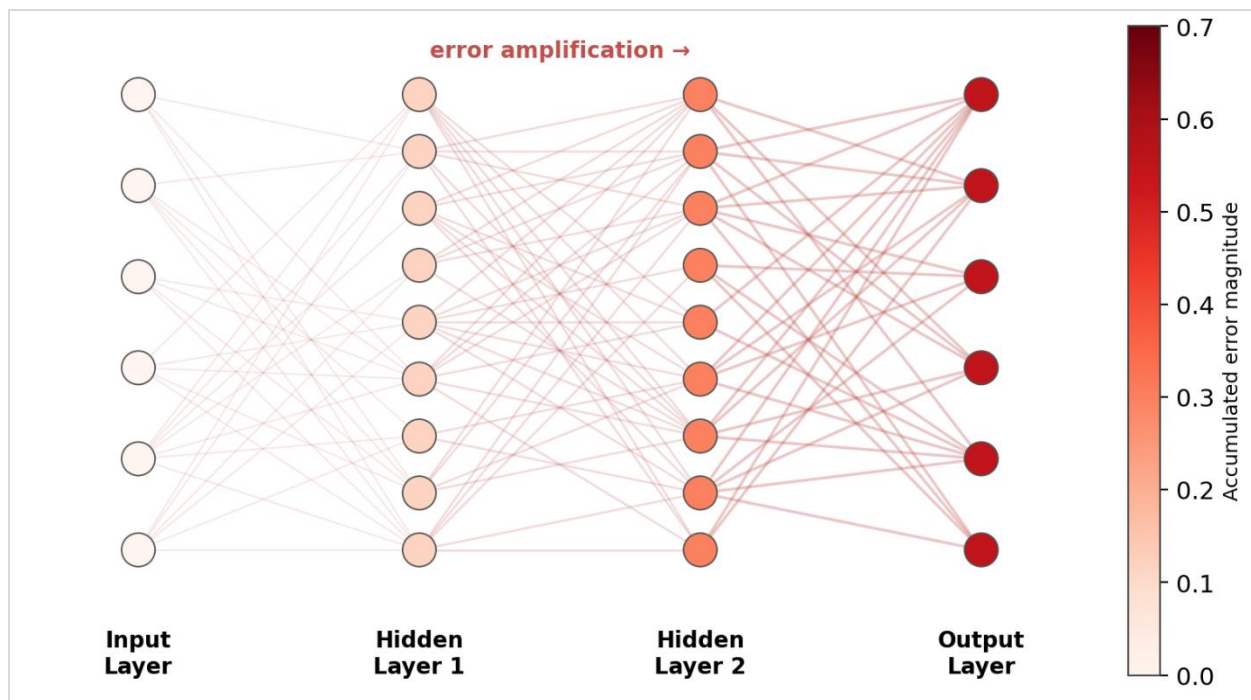
^{۴۴} Power Consumption

^{۴۵} Power-Delay Product – PDP

^{۴۶} Application-level Metrics

^{۴۷} Top-1 Accuracy

^{۴۸} Cross-Entropy Loss



شکل ۲-۴: مقایسه پویایی انتشار خطای با بایاس خنثی در برابر خطای دارای بایاس در لایه‌های شبکه عصبی عمیق.

۳. فصل سوم: ضرب‌کننده‌های

تقریبی و فشرده‌سازها

۳.۱. آناتومی عملیات ضرب در شبکه‌ی عصبی

در لایه‌های Dense شبکه‌ی مورد استفاده، خروجی هر نورون از مجموع حاصل ضرب ورودی‌ها در وزن‌های متناظر به‌علاوه‌ی بایاس تشکیل می‌شود. بنابراین هسته‌ی اصلی محاسبات این لایه، مجموعه‌ای بزرگ از عملیات Multiply-Accumulate است. اگر x_i ورودی و w_{ij} وزن اتصال ورودی i به نورون j باشد، خروجی قبل از فعال‌سازی به‌صورت زیر نوشته می‌شود:

$$y_j = \sum_i (x_i \times w_{ij}) + b_j$$

در مسیر Exact، هر عبارت $x_i \times w_{ij}$ با عملگر ضرب دقیق محاسبه می‌شود. در FPGA، ابزار HLS می‌تواند این عملیات را به منابع اختصاصی DSP نگاشت کند. مزیت این روش، حفظ مستقیم مفهوم ضرب عمومی و دقت محاسباتی است؛ اما تعداد زیاد ضرب‌ها می‌تواند مصرف DSP و توان را افزایش دهد.

۳.۲. ساختار مفهومی ضرب‌کننده‌ی دقیق

یک ضرب‌کننده‌ی دیجیتال عمومی را می‌توان به سه بخش اصلی تقسیم کرد: تولید حاصل ضرب‌های جزئی، کاهش و جمع نهایی. در تولید حاصل ضرب‌های جزئی، بیت‌های ورودی در یکدیگر ضرب منطقی می‌شوند. سپس حاصل ضرب‌های جزئی توسط شبکه‌ای از جمع‌کننده‌ها کاهش می‌یابند و در مرحله‌ی نهایی به یک مقدار واحد تبدیل می‌شوند.

در پیاده‌سازی HLS این پروژه، این ساختار منطقی به‌صورت دستی در سطح گیت طراحی نشده است؛ بلکه عبارت ضرب در سطح C++/HLS بیان شده و ابزار آن را متناسب با نوع داده، اندازه‌ی عملوندها و دستورات سنتز به منابع سخت‌افزاری نگاشت می‌کند. در گزارش سنتز مسیر Exact، ۲۵۸ منبع DSP برای هسته‌ی محاسباتی ثبت شده است که نشان می‌دهد ضرب‌های Dense به‌صورت مؤثر به منابع اختصاصی ضرب نگاشت شده‌اند.

۳.۳. محدودیت ضرب عمومی برای هدف این پژوهش

در پروژه‌ی حاضر، هدف طراحی یک ضرب‌کننده‌ی دقیق‌تر یا کوچک‌سازی مدار ضرب عمومی نیست؛ هدف اصلی بررسی این است که آیا می‌توان با محدود کردن شکل وزن‌ها، اصل عملیات ضرب را ساده کرد. این تفاوت از نظر

روشن‌شناسی مهم است: در مسیر پیشنهادی، تقریب در مقدار ضرایب مدل رخ می‌دهد و سپس معماری سخت‌افزار از همین ساختار محدودشده‌ی ضرایب استفاده می‌کند.

۳.۴. مبنای ریاضی ضرب مبتنی بر توان دو

در روش Power-of-Two، هر وزن غیرصفر به نزدیک‌ترین توان دو نگاشت می‌شود. بنابراین وزن تقریب‌زده‌شده دارای فرم زیر است:

$$w \approx w_{\text{POT}} = s \times 2^k, s \in \{-1, +1\}$$

در نتیجه، ضرب یک فعال‌سازی یا ورودی x در وزن w به صورت زیر ساده می‌شود:

$$x \times w_{\text{POT}} = s \times (x \times 2^k)$$

چون ضرب در توان دو معادل شیفت باینری است، هسته‌ی ضرب را می‌توان به Shift و کنترل علامت تبدیل کرد. این تبدیل دلیل اصلی حذف DSP در مسیر پیشنهادی است.

۳.۵. نقش Pruning در مسیر PoT/Shift

در این پروژه، PoT به صورت مستقل از Pruning استفاده نشده است. ابتدا وزن‌های کوچک یا کم‌اهمیت مطابق آستانه‌ی ۰.۱ صفر شدند و سپس وزن‌های غیرصفر به نزدیک‌ترین توان دو نگاشت شدند. این ترتیب باعث می‌شود وزن‌هایی که از ابتدا سهم محدودی دارند از مسیر محاسباتی حذف شوند و برای وزن‌های باقی‌مانده نیز نمایش بسیار ساده‌ای ایجاد شود.

$$w^P = \begin{cases} 0, & |w| \leq T \\ w, & |w| > T \end{cases}$$

در این پروژه مقدار آستانه برابر با $T=0.1$ در نظر گرفته شده است.

- نگاشت وزن‌ها به نزدیک‌ترین توان دو

پس از Pruning ، وزن‌های باقی‌مانده به نزدیک‌ترین توان دو نگاشت می‌شوند. توان متناظر با هر وزن از رابطه زیر محاسبه می‌شود:

$$k = \text{round}(\log_2(|w|))$$

و وزن تقریبی برابر است با:

$$w^{PoT} = \text{sign}(w) \times 2^k$$

- تبدیل ضرب به عملیات Shift

با توجه به نمایش وزن به صورت توان دو، عملیات ضرب را می‌توان با شیفت بیتی جایگزین کرد:

$$x \times 2^k = x \ll k$$

و برای توان منفی شیفت به راست انجام می‌دهیم:

$$x \times 2^{-k} = x \gg |k|$$

بنابراین، ضرب در مسیر پیشنهادی به ترکیبی از عملیات Shift و اعمال علامت تبدیل می‌شود.

- محاسبه خروجی نورون در مسیر Shift

خروجی نورون در مسیر Shift به صورت زیر محاسبه می‌شود:

$$y_j^{Shift} = \sum_{i=1}^N \text{sign}(w_{ij})(x_i \ll k_{ij}) + b_j$$

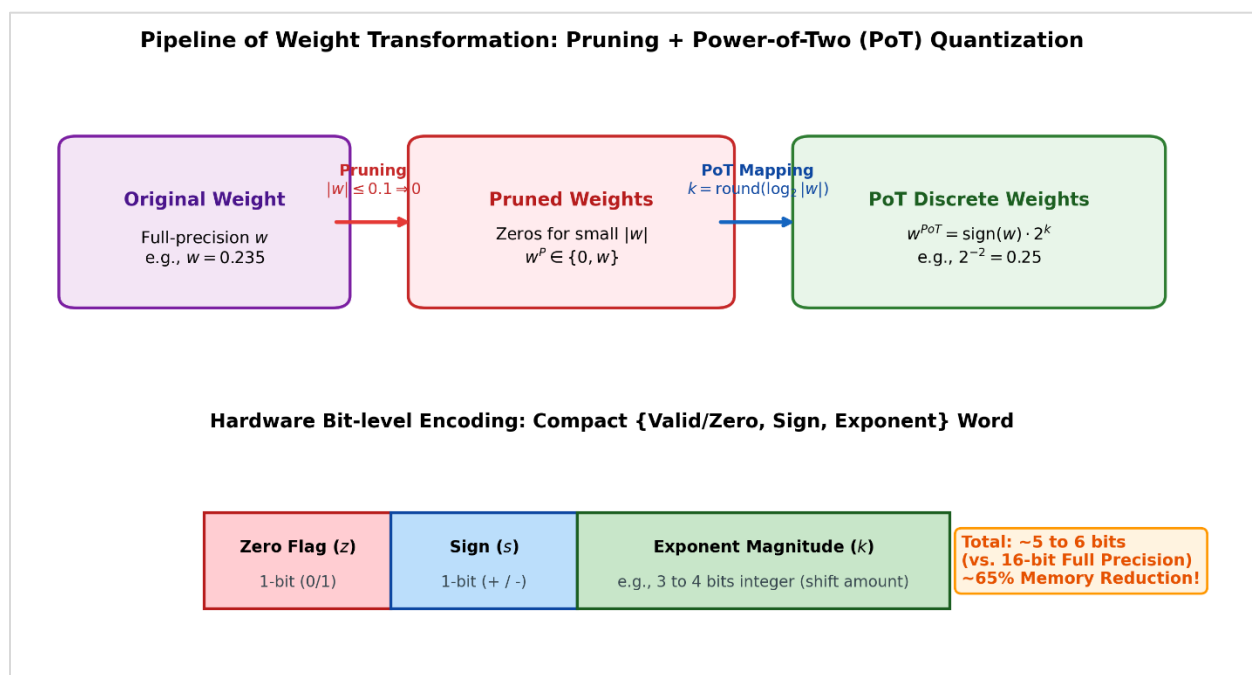
که در آن k_{ij} توان متناظر با وزن w_{ij} است.

۳.۶. نمایش سخت‌افزاری وزن‌های PoT

در مسیر سخت‌افزاری، لازم نیست مقدار اعشاری کامل وزن PoT ذخیره شود. از آنجا که هر وزن غیرصفر تنها از یک علامت و یک توان تشکیل شده است، می‌توان اطلاعات آن را به صورت sign و exponent نگهداری کرد. برای وزن صفر نیز مسیر محاسباتی می‌تواند مستقیماً مقدار صفر تولید کند. این روش هم ذخیره‌سازی ضریب را ساده می‌کند و هم مسیر محاسباتی را برای HLS قابل پیش‌بینی‌تر می‌سازد.

روند تبدیل ضرایب شبکه و فشرده‌سازی اطلاعات آن‌ها در شکل ۱-۳ خلاصه شده است. در مرحله نخست، وزن‌های با قدرمطلق کمتر از $T = 0.1$ حذف (هرس) می‌شوند.

سپس مقادیر باقی‌مانده به صورت $w^{PoT} = \text{sign}(w) \cdot 2^k$ بازنمایی می‌گردند. در لایه سخت‌افزاری، این ساختار امکان ذخیره‌سازی فشرده وزن‌ها را تنها در قالب چند بیت محدود (بیت وضعیت صفر، بیت علامت، و توان k) فراهم می‌آورد که علاوه بر حذف ضرب‌کننده‌ها، پهنای‌باند و خطوط حافظه را نیز به میزان چشمگیری ساده‌سازی می‌کند.



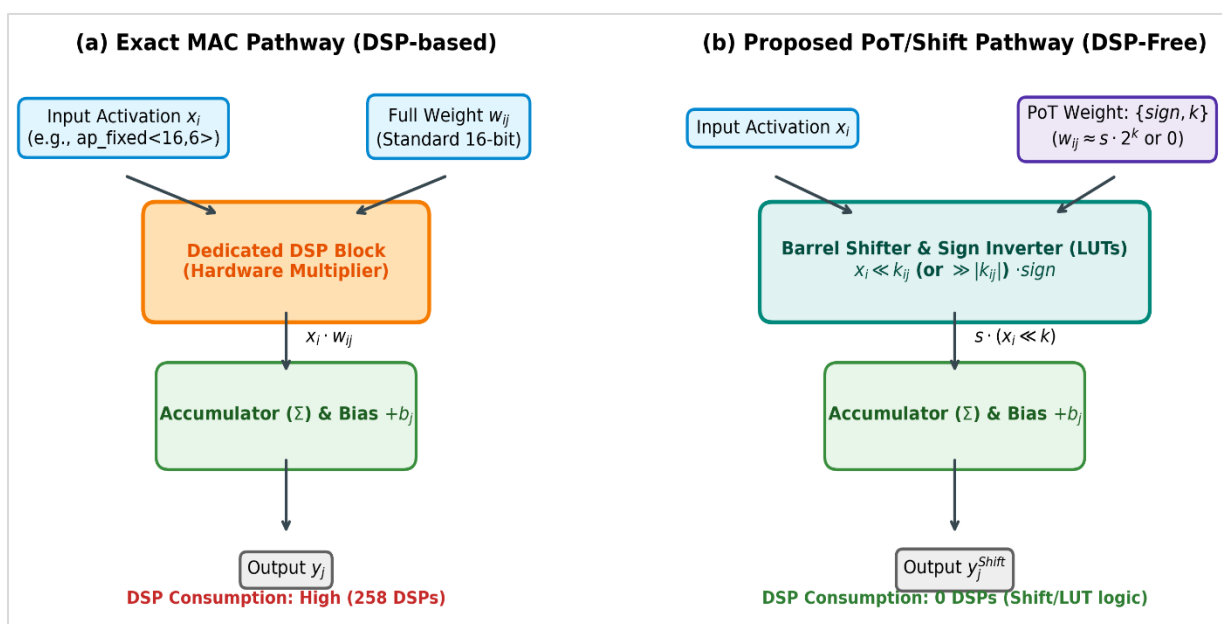
شکل ۱-۳: خط‌لوله اعمال هرس وزن‌ها، نگاشت به توان دو (PoT) و نحوه کدگذاری فشرده ضرایب در قالب کلمات بیتی سخت‌افزار

۳.۷. مقایسه‌ی مفهومی مسیر Exact و PoT/Shift

در مسیر Exact، پیاده‌سازی لایه‌ی Dense طبق رابطه‌ی کلاسیک $y = \sum (x_i \times w_i) + b$ انجام می‌شود که مستلزم استفاده از ضرب‌کننده‌های عمومی^{۴۹} است. در حالی که ساختار لایه و مرحله‌ی انباشت در هر دو رویکرد حفظ می‌شود، تفاوت اصلی در پیاده‌سازی هسته‌ی ضرب نهفته است.

این جایگزینی، ماهیت معاوضه منابع^{۵۰} را به چالش می‌کشد. از یک سو، حذف ضرب‌کننده‌های عمومی به‌طور مستقیم وابستگی طرح به بلوک‌های DSP را کاهش می‌دهد؛ اما از سوی دیگر، انتقال بار محاسباتی به سمت شیفت‌دهنده‌ها و منطق کنترل، منجر به افزایش مصرف LUTها و FFها می‌شود. بنابراین، تحلیل صرفه‌جویی سخت‌افزاری نباید صرفاً بر حذف DSPها متمرکز باشد؛ بلکه باید هزینه‌ی سربار منطق جایگزین را نیز به‌دقت در نظر گرفت.

همان‌طور که در شکل ۲-۳ نشان داده شده است، در مسیر Exact ورودی‌ها و وزن‌ها مستقیماً وارد بلوک‌های اختصاصی DSP می‌شوند که دقت ۱۰۰٪ را تضمین می‌کند اما منابع DSP را به‌شدت اشغال می‌نماید. در مقابل، در مسیر پیشنهادی PoT/Shift، هسته‌ی ضرب‌کننده با یک ترکیب بهینه از Barrel Shifter و معکوس‌کننده علامت جایگزین شده است که با استفاده از منطق LUT، نیاز به DSP را عملاً به صفر می‌رساند.



شکل ۲-۳: مقایسه معماری مسیر Exact و مسیر PoT/Shift

^{۴۹} General Multiplier

^{۵۰} Resource Trade-off

۴. فصل چهارم :هوش مصنوعی،

شبکه عصبی و طبقه‌بندی

تصویر

۴.۱. ضرورت و جایگاه شبکه‌های عصبی در طبقه‌بندی تصویر

شبکه‌های عصبی مصنوعی یکی از ابزارهای اصلی یادگیری ماشین برای حل مسائل طبقه‌بندی هستند. در یک مسئله طبقه‌بندی تصویر، هدف این است که نگاشتی از داده‌های تصویری به یکی از کلاس‌های از پیش تعیین شده یاد گرفته شود. انتخاب معماری شبکه به ویژگی‌های مجموعه داده، پیچیدگی مسئله و همچنین محدودیت‌های محاسباتی سیستم مقصد وابسته است.

در این پروژه، به جای استفاده از یک معماری پیچیده کانولوشنی، از یک شبکه عصبی سبک برای مجموعه داده MNIST استفاده شده است. دلیل این انتخاب، فراهم کردن یک محیط کنترل شده برای مطالعه اثر عملیات ضرب دقیق و تقریبی است. در این ساختار، بخش عمده محاسبات دارای پارامتر قابل آموزش مربوط به لایه‌های لایه تمام متصل^{۵۱} است و بنابراین عملیات ضرب جمع به صورت شفاف قابل بررسی و مقایسه با ضرب کننده سخت افزاری پیشنهادی است.

از دیدگاه سخت افزاری، لایه‌های Dense به تعداد زیادی عملیات ضرب بین ورودی‌ها و وزن‌ها نیاز دارند. بنابراین حتی در یک مدل سبک نیز می‌توان اثر جایگزینی ضرب کننده دقیق با ضرب کننده تقریبی را در سطح عددی و سپس در سطح سخت افزاری بررسی کرد. این ویژگی، مدل MNIST مورد استفاده در پروژه را به یک بستر آزمون مناسب برای ارزیابی رفتار ضرب کننده تبدیل می‌کند.

۴.۲. معرفی دیتاست

دیتاست MNIST^{۵۲} یکی از معروف ترین دیتاست ها در یادگیری ماشین و پردازش تصویر است. MNIST شامل تصاویر خاکستری از ارقام دست‌نویس ۰ تا ۹ است. هر تصویر دارای ابعاد ۲۸×۲۸ پیکسل است؛ بنابراین هر تصویر در مجموع دارای ۷۸۴ پیکسل است. هر تصویر به یک برچسب بین ۰ تا ۹ اختصاص داده شده که نشان دهنده ی عدد موجود در تصویر است.

^{۵۱} Fully Connected / Dense

^{۵۲} Modified National Institute of Standards and Technology

داده ها به دو بخش آموزشی^{۵۳} و تست تقسیم می شوند که در مرحله آموزش، مدل از داده های آموزشی، الگو ها و رفتار هر کلاس را یاد می گیرد و برای تست دقت مدل از داده های تست که قبلا مدل آن ها را ندیده است استفاده می شود.

مقدار	مشخصه
۷۰۰۰۰	تعداد کل تصاویر
۶۰۰۰۰	تعداد تصاویر آموزشی
۱۰۰۰۰	تعداد تصاویر آزمون
۱۰ کلاس (۰ تا ۹)	تعداد کلاس ها
۲۸ × ۲۸	ابعاد هر تصویر
تک کاناله، خاکستری	نوع تصویر
تقسیم مقادیر پیکسل بر ۲۵۵	پیش پردازش

۴.۳. لایه های مدل

شبکه عصبی مورد استفاده در این پروژه از چند نوع لایه اصلی شامل تابع فعال ساز ReLU^{54} ، لایه مسطح سازی^{۵۵} و لایه های Dense تشکیل شده است. هر یک از این لایه ها وظیفه مشخصی در استخراج ویژگی و طبقه بندی تصاویر دارند.

^{۵۳} Training

^{۵۴} Rectified Linear Unit

^{۵۵} Flatten

۴.۳.۱. لایه ورودی

تصاویر سطوح خاکستری دارای شدت روشنایی بین ۰ و ۲۵۵ هستند که در آموزش مدل آن ها را با تقسیم بر ۲۵۵ به بازه‌ی ۰ و ۱ نرمال کرده‌ایم. سپس این مقادیر نرمال شده وارد مدل می‌شود.

$$x_{\text{norm}} = \frac{x}{255}$$

۴.۳.۲. لایه مسطح‌سازی

ورودی هر نمونه یک تصویر خاکستری با ابعاد 28×28 است. لایه Flatten هیچ پارامتر قابل آموزش ندارد و فقط این ماتریس دوبعدی را به یک بردار یک‌بعدی تبدیل می‌کند. بنابراین، هر تصویر پس از Flatten به یک بردار ۷۸۴ عضوی تبدیل می‌شود و این بردار ورودی لایه Dense اول خواهد بود.

۴.۳.۳. لایه تمام‌متصل^{۵۶} اول

در یک لایه Dense، هر خروجی از جمع وزن‌دار ورودی‌ها به‌همراه بایاس حاصل می‌شود. رابطه کلی برای خروجی نرون j ام به صورت زیر است:

$$y_j = \sum_{i=1}^n x_i \cdot w_{ij} + b_j$$

که در آن x_i ورودی، w_{ij} وزن اتصال ورودی i به نرون j ، b_j بایاس نرون j و y_j خروجی نرون است. این رابطه نشان می‌دهد که هسته اصلی محاسبات Dense شامل ضرب و جمع است و همین بخش مستقیماً به موضوع طراحی ضرب‌کننده دقیق و تقریبی مرتبط می‌شود.

^{۵۶} Fully Connected / Dense

لایه Dense اول دارای ۷۸۴ ورودی و ۱۲۸ نورون خروجی است. بنابراین تعداد پارامترهای آن برابر است با:

$$\text{Parameters Dense}_1 = (784 \times 128) + 128 = 100,480$$

پس از لایه Dense اول از تابع فعال‌ساز غیر خطی^{۵۷} استفاده می‌شود. این تابع مقادیر منفی را به صفر تبدیل کرده و مقادیر مثبت را بدون تغییر عبور می‌دهد:

$$\text{ReLU}(x) = \max(0, x)$$

وجود ReLU باعث ایجاد غیرخطی بودن شبکه می‌شود و توانایی آن را برای یادگیری روابط پیچیده‌تر افزایش می‌دهد. از دیدگاه تحلیل خطای ضرب، خروجی‌های منفی که توسط ReLU حذف می‌شوند می‌توانند بر نحوه انتشار برخی خطاهای عددی اثر بگذارند؛ با این حال، میزان واقعی تحمل خطا باید با آزمایش تعیین شود.

۴.۳.۴. لایه حذف کردن^{۵۸}

برای کاهش بیش‌پردازش، پس از لایه Dense اول از Dropout با نرخ ۰/۲ استفاده شده است. در زمان آموزش، بخشی از نورون‌ها به صورت تصادفی غیرفعال می‌شوند تا شبکه بیش از حد به مسیرهای مشخص وابسته نشود. Dropout در مرحله آموزش فعال است و در مرحله interface غیرفعال می‌شود. بنابراین در تحلیل سخت‌افزاری مدل، عملیات ضرب اصلی مربوط به Dense‌هاست و Dropout در مسیر استنتاج نقش محاسباتی مستقیم ندارد.

۴.۳.۵. لایه تمام‌متصل^{۵۹} دوم

لایه Dense خروجی دارای ۱۰ نورون است که هر نورون متناظر با یکی از ارقام ۰ تا ۹ است که نشان دهنده‌ی کلاس پیش‌بینی شده است.

^{۵۷} ReLU

^{۵۸} Dropout

^{۵۹} Fully Connected / Dense

لایه Dense دارای ۱۲۸ ورودی و ۱۰ نورون است و تعداد پارامترهای آن برابر است با:

$$\text{Parameters Dense}_2 = (128 \times 10) + 10 = 1,290$$

معماری نهایی مورد استفاده یک شبکه عصبی تمام‌متصل سبک است که برای طبقه‌بندی ده‌کلاسه MNIST طراحی شده است. مزیت اصلی این ساختار در پروژه حاضر، سادگی مسیر محاسباتی و امکان شناسایی مستقیم عملیات ضربی است؛ به‌گونه‌ای که می‌توان اثر جایگزینی ضرب‌کننده دقیق با ضرب‌کننده تقریبی را بدون درگیر شدن با پیچیدگی‌های اضافی یک شبکه کانولوشنی ارزیابی کرد.

شماره	نوع لایه	ابعاد خروجی	تعداد پارامتر	توضیح
۰	Input	28×28	۰	ورودی تصویر
۱	Flatten	۷۸۴	۰	تبدیل تصویر به بردار
۲	Dense	۱۲۸	۱۰۰۴۸۰	فعال‌ساز ReLU
۳	Dropout	۱۲۸	۰	نرخ ۰٫۲ در آموزش
۴	Dense	۱۰	۱۲۹۰	لایه خروجی

در نتیجه تعداد کل پارامترهای مدل برابر است با:

$$\text{Total Parameters} = 100,480 + 1,290 = 101,770$$

از این تعداد، ۱۰۱۹۶۳۲ مقدار مربوط به ماتریس‌های وزن دو لایه Dense است و ۲۵۶ مقدار مربوط به بایاس‌های این دو لایه است. بنابراین تقریباً تمام وزن‌های قابل تغییر در مسیر محاسبات ضربی قرار دارند و همین موضوع استفاده از مدل را برای ارزیابی ضرب‌کننده مناسب می‌کند.

$$N_{w,\text{total}} = 100,352 + 1,280 = 101,632$$

$$N_{w,1} = 784 \times 128 = 100,352$$

$$N_{w,2} = 128 \times 10 = 1,280$$

مدل در محیط TensorFlow/Keras آموزش داده شده است. فرآیند آموزش با بهینه‌ساز Adam و تابع خطای آنتروپی متقاطع طبقه‌بندی پراکنده^{۶۰} انجام شده و دقت طبقه‌بندی روی مجموعه آزمون به‌عنوان معیار اصلی عملکرد گزارش شده است.

پارامتر	مقدار
Framework	TensorFlow / Keras
Dataset	MNIST
Optimizer	Adam
Loss	Sparse Categorical Cross.Entropy
Epochs	۵۰
ابعاد ورودی	۲۸ × ۲۸
تعداد خروجی‌ها	۱۰

۴.۴ اعمال توان دو^{۶۱} و هرس کردن^{۶۲}

به‌منظور بررسی یک روش ساده برای کاهش پیچیدگی نمایش وزن‌ها، پس از پایان آموزش مدل پایه، یک مرحله PoT همراه با Pruning روی وزن‌های لایه‌های چندبندی اعمال شد. این عملیات در زمان آموزش اولیه وجود نداشته و وزن‌ها پس از آموزش به نزدیک‌ترین توان دو نگاشت شدند.

در مرحله Pruning، هر وزنی که قدرمطلق آن کمتر از یا مساوی ۰/۱ باشد، صفر در نظر گرفته شد. برای وزن‌های باقی‌مانده، نزدیک‌ترین توان دو انتخاب شد. به این ترتیب، نمونه‌هایی مانند ۰/۳۷ به ۰/۵ و ۰/۲۴ به ۰/۲۵ نگاشت می‌شوند و علامت وزن نیز حفظ می‌گردد.

$$w_{\text{approx}} = 2^{\text{round}(\log_2|w|)} \times \text{sgn}(w), |w| > 0.1$$

^{۶۰} sparse categorical cross-entropy

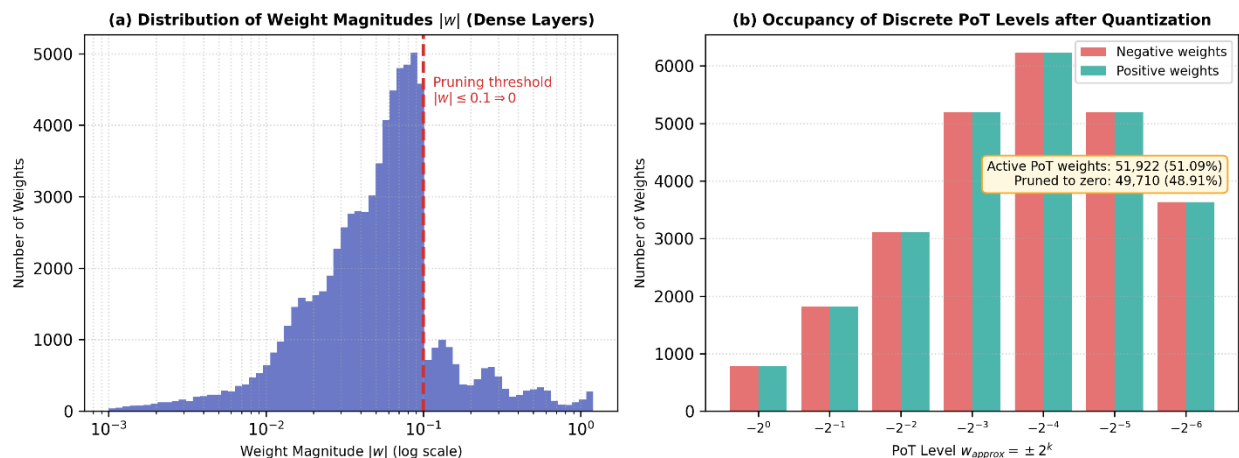
^{۶۱} Power-of-Two (PoT)

^{۶۲} Pruning

$$w_{\text{approx}} = 0, |w| \leq 0.1$$

این مدل همان مدلی است که در ادامه برای تولید ضرایب مناسب و بررسی پیاده‌سازی مبتنی بر Shift در HLS استفاده شد.

بر اساس این آزمایش، ۴۹۷۱۰ مقدار از ۱۰۱۶۳۲ مقدار وزن مورد بررسی به دلیل قرار گرفتن در زیر آستانه تعیین شده یا برابر بودن با آن صفر شده‌اند؛ این مقدار حدود ۴۸/۹۱ درصد از وزن‌های مورد پردازش را تشکیل می‌دهد. در نتیجه، بخش قابل توجهی از ضرایب کوچک حذف شده و وزن‌های باقی‌مانده نیز به مقادیر توان دو نگاشت شده‌اند. شکل ۴-۱ توزیع آماری قدرمطلق وزن‌ها قبل از هرس و میزان اشغال سطوح گسسته پس از کوانتیزاسیون را نشان می‌دهد.



شکل ۴-۱: تحلیل آماری پارامترهای وزن مدل: (الف) هیستوگرام توزیع قدرمطلق وزن‌ها به همراه محدوده هرس‌سازی با آستانه $T = 0.1$ ، (ب) فراوانی و نرخ اشغال سطوح گسسته توان‌های دو ($\pm 2^k$) پس از کوانتیزاسیون PoT

این ویژگی در مرحله سخت‌افزاری امکان استفاده از Shift را به جای ضرب عمومی فراهم کرده و مدل را برای ارزیابی منابع، توان و کارایی در HLS/Vivado آماده می‌کند.

$$\text{Pruning Ratio} = (49710 / 101,632) \times 100 \approx 48.91 \%$$

لازم است تأکید شود که PoT با کوانتیزاسیون نقطه ثابت یکسان نیست. فایل مدل Keras پس از این مرحله همچنان می‌تواند در قالب Float32 نگهداری شود؛ آنچه برای مسیر سخت‌افزاری اهمیت دارد، نمایش ضرایب PoT و نحوه استفاده از آن‌ها در محاسبات HLS است. در پیاده‌سازی انجام‌شده، ضرایب PoT به‌صورت علامت و توان ذخیره شده و در مسیر محاسباتی با نمایش نقطه‌ثابت مورد استفاده قرار گرفته‌اند. بنابراین، PoT/Pruning در این پروژه صرفاً یک آزمایش نرم‌افزاری مستقل نیست، بلکه بخشی از زنجیره واقعی آماده‌سازی مدل تا پیاده‌سازی در HLS/Vivado است.

۴.۵. اثر خطای ضرب در بخش‌های محاسباتی و تاب‌آوری خطا^{۶۳}

یکی از اهداف اصلی این فصل، آماده‌سازی یک مدل مرجع برای بررسی اثر خطای محاسباتی در مراحل سخت‌افزاری است. در مدل حاضر، عملیات ضرب به‌طور مستقیم در دو لایه Dense رخ می‌دهد و هر خروجی این لایه‌ها از مجموع حاصل ضرب ورودی و وزن تشکیل می‌شود.

در حالت مرجع، هر حاصل ضرب با ضرب‌کننده دقیق محاسبه می‌شود. در مراحل بعدی، همین عملیات می‌تواند با ضرب‌کننده تقریبی پیشنهادی جایگزین شود و خروجی مدل با حالت دقیق مقایسه گردد. در این مقایسه، معیار اصلی تنها خطای عددی یک ضرب نیست؛ بلکه باید بررسی شود که خطاهای ایجادشده پس از عبور از لایه‌ها تا چه اندازه بر پیش‌بینی نهایی اثر می‌گذارند.

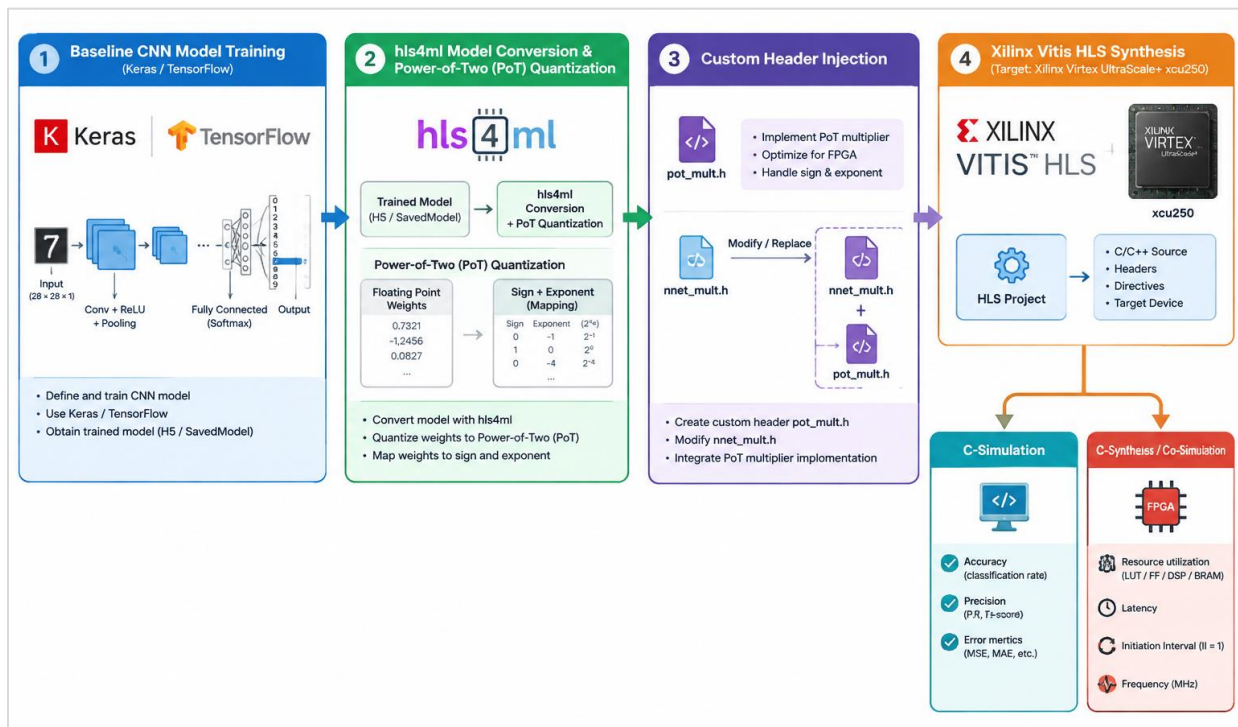
ارزیابی نهایی جایگزینی ضرب دقیق با ضرب‌کننده تقریبی پیشنهادی باید در محیط HLS/Vivado و با همان داده‌های ورودی انجام شود تا اثر واقعی تقریب سخت‌افزاری از سایر منابع خطا تفکیک گردد.

^{۶۳} Error Resilience

۵. فصل پنجم: روش پیشنهادی و پیاده‌سازی سخت‌افزاری در hls4ml

۵.۱. متدولوژی کلی و جریان کار^{۶۴}

پیاده‌سازی شبکه‌های عصبی عمیق^{۶۵} بر روی سخت‌افزارهای محدود مانند FPGA ها با هدف دستیابی به پردازش با توان پایین و تأخیر کم، نیازمند رویکردهای نوآورانه در زمینه محاسبات تقریبی و سنتز خودکار سخت‌افزار است. چارچوب hls4ml به عنوان یک ابزار قدرتمند، پل ارتباطی میان محیط‌های یادگیری ماشین سطح بالا مانند TensorFlow/Keras و ابزارهای سنتز سطح بالا (High-Level Synthesis - HLS) برای FPGA ها عمل می‌کند. این فصل به تشریح متدولوژی کلی پژوهش، جریان کار پیاده‌سازی، جزئیات کوانتیزه‌سازی، طراحی ضرب‌کننده سفارشی مبتنی بر توان دو و استراتژی‌های بهینه‌سازی HLS می‌پردازد. تصویر ۵-۱ این جریان کار را به صورت خلاصه نمایش می‌دهد.



تصویر ۵-۱: نمای کلی جریان کار متدولوژی پیشنهادی از آموزش مدل پایه در Keras/TensorFlow تا سنتز و ارزیابی روی تراشه

^{۶۴} Design Workflow

^{۶۵} Deep Neural Networks - DNNs

۵.۱.۱. گام اول: طراحی و آموزش مدل پایه در Keras/TensorFlow

اولین گام، طراحی و آموزش یک مدل CNN پایه با استفاده از کتابخانه‌های TensorFlow و Keras بود. معماری شبکه متناسب با مجموعه داده مورد استفاده انتخاب شد. هدف این مرحله، دستیابی به یک مدل پایه با بالاترین دقت ممکن در حالت محاسبات دقیق بود. پس از آموزش و اعتبارسنجی، ساختار مدل و پارامترهای آن (وزن‌ها و بایاس‌ها) به عنوان نقطه مرجع (Baseline) استخراج و ذخیره شدند.

۵.۱.۲. گام دوم: تبدیل مدل و نگاشت به نمایش توان دو (PoT) با hls4ml

مدل آموزش‌دیده به چارچوب hls4ml وارد شد. وظیفه hls4ml تبدیل کد مدل پایتونی به کدهای C++ قابل فهم برای ابزارهای HLS است. چالش کلیدی در این مرحله، پیاده‌سازی محاسبات تقریبی بود. برخلاف رویکردهای رایج کوانتیزه‌سازی به نقطه ثابت (Fixed-Point)، در این پژوهش، وزن‌ها و گاهی فعال‌سازی‌ها به نمایش توان دو (Power-of-Two - PoT) نگاشت شدند. این نمایش، وزن‌ها را به صورت $w = \text{sign} \times 2^{\text{exponent}}$ بیان می‌کند، که عملیات ضرب را به عملیات شیفت ساده تبدیل می‌کند. این نگاشت با دقت از پیش تعیین‌شده انجام شد تا از سرریز محاسباتی جلوگیری شود و پیچیدگی منابع سخت‌افزاری کاهش یابد.

۵.۱.۳. گام سوم: بازطراحی هدرهای حسابی و تزریق ضرب‌کننده سفارشی

چارچوب hls4ml به طور پیش‌فرض عملیات ریاضی (ضرب، جمع و غیره) را با استفاده از عملگرهای استاندارد C++ یا کتابخانه‌های نقطه ثابت پیاده‌سازی می‌کند. برای اعمال تقریب محاسباتی مبتنی بر PoT، لازم بود که هسته‌های محاسباتی اصلی (مانند ضرب ماتریس در لایه‌های تمام‌متصل) بازطراحی شوند. این کار با اصلاح هدرهای کتابخانه داخلی hls4ml انجام شد.

۵.۱.۴. گام چهارم: سنتز، ارزیابی منابع و شبیه‌سازی در Vitis HLS

پروژه C++ تولیدشده توسط hls4ml و اصلاح‌شده با کد سفارشی PoT، به محیط Xilinx Vitis HLS وارد شد. در این محیط، دو نوع ارزیابی صورت گرفت:

- **شبیه‌سازی تابعی (C Simulation):** با استفاده از داده‌های آزمون، عملکرد الگوریتمی مدل تقریبی سنجیده شد تا میزان افت دقت نسبت به مدل پایه مشخص گردد.

- سنتز سطح بالا (C Synthesis): کد ++C با هدف قرار دادن تراشه مشخص (Xilinx Virtex UltraScale+ xcu250) سنتز شد. گزارش‌های حاصل از این مرحله شامل معیارهای سخت‌افزاری کلیدی مانند تأخیر (Latency)، فاصله شروع (Initiation Interval – II)، فرکانس کاری بیشینه (Maximum Frequency – Fmax) و میزان مصرف منابع تراشه (LUTs, Flip-Flops, DSP blocks, BRAMs) استخراج و تحلیل گردید.

۵.۲. چارچوب hls4ml

hls4ml یک چارچوب متن‌باز^{۶۶} است که فرآیند طراحی شتاب‌دهنده‌های یادگیری ماشین بر روی FPGAها را تسهیل می‌کند. این ابزار با هدف کاهش پیچیدگی فرآیند “کد-طراحی-تراشه”^{۶۷} ایجاد شده است.

ویژگی‌های کلیدی hls4ml:

- پشتیبانی از مدل‌های محبوب: hls4ml قادر به وارد کردن مدل‌های آموزش‌دیده از چارچوب‌های یادگیری عمیق رایج مانند Keras/TensorFlow، PyTorch، و scikit-learn است.
- تبدیل خودکار به HLS: این چارچوب به طور خودکار مدل پایتونی را به کدهای ++C یا Verilog/VHDL تبدیل می‌کند که برای ابزارهای HLS مانند Vitis HLS یا Vivado HLS قابل استفاده باشند.
- مدیریت کوانتیزه‌سازی: hls4ml از کوانتیزه‌سازی خودکار وزن‌ها و فعال‌سازی‌ها به فرمت‌های نقطه ثابت یا باینری پشتیبانی می‌کند. این قابلیت برای کاهش مصرف منابع سخت‌افزاری حیاتی است.
- قابلیت سفارشی‌سازی: کاربران می‌توانند هسته‌های محاسباتی (مانند ضرب‌کننده‌ها) و استراتژی‌های کوانتیزه‌سازی را مطابق با نیازهای خود سفارشی‌سازی کنند، که این امکان در پژوهش حاضر مورد استفاده قرار گرفت.

^{۶۶} Open-Source

^{۶۷} Code-Design-Chip

- بهینه‌سازی **HLS**: **hls4ml** با اعمال خودکار دستورالعمل‌ها (Pragmas/Directives) به کد ++C تولیدشده، به ابزارهای HLS کمک می‌کند تا بهترین عملکرد و کمترین مصرف منابع را برای FPGA هدف استخراج کنند.

- پشتیبانی از پلتفرم‌های **FPGA**: این چارچوب با طیف وسیعی از پلتفرم‌های FPGA از سازندگان مختلف (مانند Intel و Xilinx) سازگار است.

استفاده از **hls4ml** در این پژوهش، امکان تمرکز بر جنبه‌های نوآورانه محاسبات تقریبی (یعنی طراحی **pot_mult**) را فراهم آورد، در حالی که پیچیدگی‌های مربوط به تبدیل مدل و تولید کد HLS به این چارچوب سپرده شد.

۵.۳. مدل‌سازی عددی و کوانتیزه‌سازی به توان دو (PoT)

همانطور که پیشتر ذکر شد، هدف اصلی این پژوهش، جایگزینی عملیات ضرب محاسباتی با عملیات شیفت بیتی بود. این امر مستلزم نگاشت وزن‌های مدل به نمایش توان دو بود. وزن‌ها در نمایش PoT به صورت $w = 2^e \times S$ بیان می‌شوند، که در آن S علامت (مثبت یا منفی) و e نمای (exponent) وزن است.

در پیاده‌سازی انجام شده، از ساختار `<W, I>ap_fixed` برای نمایش مقادیر ورودی و خروجی محاسبات استفاده شد، که در آن W تعداد کل بیت‌ها و I تعداد بیت‌های صحیح است. اما خود وزن‌ها (که در `nnet::product::pot_mult` به کار می‌روند) به صورت یک ساختار سفارشی حاوی سه مؤلفه ذخیره شدند:

- **nonzero**: یک بیت منطقی که نشان می‌دهد آیا وزن غیرصفر است یا خیر. اگر وزن صفر باشد، کل حاصل ضرب صفر خواهد بود و نیازی به محاسبه نیست.
- **positive**: یک بیت منطقی که علامت وزن را مشخص می‌کند.
- **exponent**: یک عدد صحیح کوچک (مانند `<4>ap_int`) که نمای e را نشان می‌دهد.

تابع `pot_mult` با دریافت ورودی a و وزن w (که شامل `exponent`, `positive`, `nonzero` است)، ابتدا بررسی می‌کند که آیا وزن غیرصفر است. اگر غیرصفر بود، ورودی a را به تعداد بیت‌های مشخص شده توسط

w.exponent شیفت می‌دهد ($w.exponent < x$). سپس، اگر w.positive نادرست بود، نتیجه را منفی می‌کند.

این روش، ضمن حذف کامل نیاز به واحدهای DSP برای عملیات ضرب، پیچیدگی محاسباتی را به شدت کاهش می‌دهد. البته، این نمایش گسسته وزن‌ها، دقت مدل را کاهش می‌دهد، که این افت دقت در فصل نتایج (فصل ۶) به تفصیل تحلیل شده است.

۵.۴. تزریق ضرب‌کننده سفارشی PoT به hls4ml

برای جایگزینی تابع ضرب پیش‌فرض در hls4ml با عملیات مبتنی بر شیفت، تغییرات زیر در کد منبع اعمال شد:

1. ایجاد هدر **pot_mult.h**: یک فایل هدر جدید با نام **pot_mult.h** ایجاد گردید که حاوی کلاس **nnet::product::pot_mult** است. این کلاس، تابع استاتیک **product** را پیاده‌سازی می‌کند که دو آرگومان ورودی (یک ورودی شبکه و یک وزن) را گرفته و حاصل ضرب تقریبی را محاسبه و بازمی‌گرداند.
2. اصلاح **nnet_mult.h**: فایل اصلی **nnet_mult.h** که توابع ضرب در **hls4ml** را تعریف می‌کند، به گونه‌ای تغییر یافت که به جای فراخوانی تابع ضرب پیش‌فرض، تابع **nnet::product::pot_mult::product** را فراخوانی کند. این تغییر به صورت سراسری بر تمام عملیات ضرب در لایه‌های تمام‌متصل اعمال شد.
3. مدیریت نمایش وزن: وزن‌های مدل Keras که در ابتدا به صورت تنسورهای **float32** بودند، پس از تبدیل توسط **hls4ml** به فرمت مناسب (مثلاً **ap_fixed** یا **ap_int**) نگاشت شدند. سپس، در فرآیند تبدیل، این مقادیر به ساختار سفارشی PoT (شامل **exponent**, **positive**, **nonzero**) تبدیل و ذخیره شدند تا توسط تابع **pot_mult** قابل استفاده باشند.

در این روش از قبل در کد پایتون هنگام تولید مدل **pot** وزن‌های تولید شده و تعداد آن‌ها رو بررسی کردیم که وزن‌ها توان‌هایی در بازه منفی سه تا یک تولید می‌کردند که بیشترین وزن‌های تولید شده به توان منفی

سه تقریب زده شدند. ابتدا همه‌ی توان‌ها در تعریف تابع استفاده کردیم و سپس برای کاهش LUT و FF توان‌هایی که تعداد کم‌تری در وزن‌های تولید شده داشتند را حذف کردیم.

۵.۵. استراتژی‌های بهینه‌سازی

برای دستیابی به بهترین عملکرد سخت‌افزاری (کمترین تأخیر و مصرف منابع)، از دستورالعمل‌های (Directives) استاندارد Vitis HLS استفاده شد. این دستورالعمل‌ها به ابزار HLS کمک می‌کنند تا کد ++C را به گونه‌ای به کد RTL (Register-Transfer Level) تبدیل کند که بهینه باشد. برخی از کلیدی‌ترین دستورالعمل‌های مورد استفاده عبارتند از:

- `pragma HLS PIPELINE II=۱`: برای فعال‌سازی پایپ‌لاینینگ در توابع و حلقه‌ها، با هدف کاهش فاصله شروع (Initiation Interval) به ۱ چرخه کلاک. این دستورالعمل برای دستیابی به توان عملیاتی (Throughput) بالا حیاتی است.
- `<N>=pragma HLS UNROLL factor#`: برای باز کردن (Unroll) حلقه‌ها تا فاکتور N ، که باعث موازی‌سازی عملیات و کاهش تأخیر کلی می‌شود.
- `complete/cyclic/block <name>=pragma HLS ARRAY_PARTITION variable#`: برای پارتیشن‌بندی آرایه‌ها و حافظه‌ها (مانند وزن‌ها یا فعال‌سازی‌های میانی)، به منظور افزایش پهنای باند دسترسی و امکان موازی‌سازی بیشتر.
- `pragma HLS INTERFACE#`: برای تعریف رابط‌های ورودی/خروجی ماژول HLS و تعیین نوع پروتکل ارتباطی (مانند AXI-Stream).
- انتخاب و تنظیم دقیق این دستورالعمل‌ها بر اساس ساختار شبکه و محدودیت‌های تراشه هدف (xcu۲۵۰) صورت پذیرفت تا تعادل بهینه‌ای میان تأخیر، توان عملیاتی و مصرف منابع برقرار گردد.

۵.۶. تفکیک شبیه‌سازی نرم‌افزاری (C.Sim) و سنتز سخت‌افزاری (C.Synthesis)

در فرآیند ارزیابی، تمایز دقیقی میان نتایج حاصل از شبیه‌سازی نرم‌افزاری (C Simulation) و سنتز سخت‌افزاری (C Synthesis) رعایت گردید:

- شبیه‌سازی (C Simulation) ++C: این مرحله، عملکرد الگوریتمی مدل تقریبی را با استفاده از داده‌های واقعی ارزیابی می‌کند. خروجی‌های اصلی این شبیه‌سازی شامل معیارهای دقت طبقه‌بندی (Accuracy)، Recall، Precision و تحلیل توزیع خطا یا نویز در سطح داده‌ها است. این شبیه‌سازی به ما اطمینان می‌دهد که تقریب اعمال شده، باعث افت شدید عملکرد الگوریتمی نشده است.
- سنتز سطح بالا (C Synthesis / Co-Simulation): این مرحله، کد ++C را به زبان RTL (مانند Verilog) تبدیل کرده و گزارشی از منابع سخت‌افزاری مصرفی (LUT, FF, DSP, BRAM) و مشخصات عملکردی مدار سنتز شده (تأخیر، فرکانس، II) ارائه می‌دهد. شبیه‌سازی هم‌زمان (CO-Simulation) صحت عملکرد مدار سنتز شده را با ورودی‌های شبیه‌سازی ++C تأیید می‌کند. این بخش، اطلاعات حیاتی برای ارزیابی هزینه سخت‌افزاری و محدودیت‌های عملکردی پیاده‌سازی فراهم می‌آورد.

این تفکیک، امکان تحلیل جامع از دیدگاه‌های الگوریتمی و سخت‌افزاری را فراهم می‌آورد و به ما اجازه می‌دهد تا تأثیر تقریب را هم بر دقت مدل و هم بر منابع سخت‌افزاری به طور دقیق بررسی کنیم.

۶. فصل ششم: آزمایش‌ها، نتایج و تحلیل

۶.۱. چارچوب آزمون تجربی و تنظیمات پیاده‌سازی

ارزیابی تجربی یک شتاب‌دهنده سخت‌افزاری نیازمند بررسی هم‌زمان پایداری الگوریتمی و بهره‌وری فیزیکی مدار است. در این پژوهش، مدل شبکه عصبی کانولوشنی نگاشت‌شده روی مجموعه داده استاندارد MNIST، از نظر دقت، بازیابی^{۶۸}، صحت^{۶۹} و پارامترهای سنتز مدار مورد بررسی قرار گرفت. تمامی ارزیابی‌های سنتز سطح بالا^{۷۰} در محیط ۲۰۲۳.۲ Vitis HLS و با هدف‌گذاری روی پلتفرم پیشرفته Xilinx Virtex UltraScale+ (xcu۲۵۰-figd۲۱۰۴-۲L-e) به انجام رسیده است.

به منظور نمایش شفاف فرآیند طراحی، نتایج در دو بخش اصلی بررسی می‌شوند:

۱. سنجش الگوریتمی مدل (دقت، صحت و بازخوانی در مقایسه با نسخه دقیق).

۲. سیر تکامل سخت‌افزاری در ۳ گام بهینه‌سازی متوالی (پیاده‌سازی اولیه، بهینه‌سازی میانی، و معماری نهایی کمینه‌شده).

۶.۲. سنجش عملکرد الگوریتمی

در ارزیابی مدل‌های طبقه‌بندی، صرف تکیه بر شاخص Accuracy به دلیل احتمال نادیده گرفتن توزیع خطای کلاس‌ها کافی نیست. از این رو، بردارهای پیش‌بینی روی ۱۰,۰۰۰ تصویر آزمون MNIST با معیارهای تفصیلی سنجیده شدند:

جدول ۶-۱: ارزیابی تطبیقی کارایی الگوریتمی مدل بر روی دیتاست MNIST

شاخص عملکردی	مدل دقیق مبنا	مدل تقریبی پیشنهادی	میزان تغییرات	تحلیل پایداری شبکه
دقت کلی (Accuracy)	۹۵.۹۹	۹۳.۷۸	-۲.۲۱٪	حفظ پایداری استنتاج و عدم واگرایی فیلترها

^{۶۸} Recall

^{۶۹} Precision

^{۷۰} C-Synthesis

صحت میانگین (Precision)	۹۶.۱۳٪	۹۴.۰۹٪	-۲.۰۴٪	نرخ پایین اعلام مثبت کاذب ^{۷۱}
بازخوانی میانگین (Recall)	۹۵.۹۰٪	۹۳.۶۴٪	-۲.۲۶٪	شناسایی موفق ارقام و حداقل منفی کاذب

تحلیل مقایسه‌ای مقادیر فوق نشان می‌دهد که افت در تمامی شاخص‌های ارزیابی، به‌صورت کاملاً متوازن و همگن رخ داده است. به‌طور خاص:

۱. **تداوم دقت و صحت:** کاهش ۲.۰۴ درصدی در Precision نشان می‌دهد که مدل تقریبی همچنان توانایی تفکیک کلاس‌ها را با درصد خطای بسیار محدود حفظ کرده و تقریب لایه‌های ضرب موجب پاشش کلاس‌ها^{۷۲} نشده است.

۲. **حفظ حساسیت مدل:** افت ۲.۲۶ درصدی در Recall حاکی از آن است که مدل با وجود استفاده از محاسبات تقریبی، همچنان قادر به شناسایی و بازیابی ارقام از مجموعه داده ورودی با حساسیت بالاست و در بازشناسایی الگوها دچار کورسویی^{۷۳} نسبت به ارقام خاص نشده است.

نتیجه‌گیری فنی:

افت همگن و بسیار اندک (در محدوده ۰.۲٪) در هر سه شاخص، اثبات می‌کند که استراتژی تقریب اتخاذ شده در ساختار ضرب‌کننده‌های لایه‌های متراکم و کانولوشن، هیچ‌گونه اریب (Bias) یا انحراف سیستماتیک روی کلاس‌های خاصی از MNIST ایجاد نکرده است. در واقع، اثر محاسبات تقریبی صرفاً به یک نویز با واریانس پایین در فضای برداری ویژگی‌ها^{۷۴} محدود شده است که شبکه عصبی توانایی پذیرش و فیلتر کردن آن را بدون افت شدید کارایی داراست.

۶.۳. سیر تکامل سلسله‌مراتبی و فازهای بهینه‌سازی سخت‌افزاری

^{۷۱} False Positives

^{۷۲} Class Dispersion

^{۷۳} Blindness

^{۷۴} Feature Space

یکی از دستاوردهای بارز این پژوهش، فرآیند پالایش گام به گام معماری تقریبی برای مهار مصرف جداول جستجو (LUT) بود. فرآیند بهینه سازی در سه گام متوالی پیاده سازی شد:

- **گام ۱ (پیاده سازی اولیه تقریب^{۷۵}):** جایگزینی ضرب کننده ها با ساختار تقریبی اولیه و حذف کامل DSP ها. در این مرحله اگرچه وابستگی به DSP قطع شد، اما مصرف LUT به دلیل افزودن در ساختار عبارات حسابی به ۱۶۱,۰۷۸ رسید (۳۷٪ یک SLR).
- **گام ۲ (بهینه سازی میانی منطق^{۷۶}):** با فاکتورگیری از عبارات منطقی مشترک در لایه های Dense و بازآرایی تسهیم کننده ها، مصرف LUT با ۱۶.۳٪ بهبود به ۱۳۴,۸۷۹ تقلیل یافت.
- **گام ۳ (طراحی نهایی کمینه شده^{۷۷}):** با بهینه سازی و بازچینی مدار نگاشت بیتی، مصرف LUT به ۹۸,۵۱۹ کاهش یافت (۳۸.۸٪ کاهش مساحت منطقی نسبت به گام اولیه تقریب).

۶.۴. ارزیابی جامع نتایج سنتز و مصرف منابع فیزیکی

گزارش های رسمی استخراج شده از C-Synthesis ابزار Vitis HLS در جدول ۶-۲ به صورت تطبیقی گردآوری شده است.

جدول ۶-۲: ارزیابی تطبیقی سنتز فیزیکی بین مدل دقیق مبنا و گام های سه گانه معماری تقریبی

شاخص ارزیابی / منبع سخت افزاری	مدل دقیق مبنا (Exact)	گام ۱: تقریب اولیه	گام ۲: بهینه سازی میانی	گام ۳: نسخه نهایی پیشنهادی	تغییرات نهایی نسبت به دقیق	مفهوم و دستاورد مهندسی
بلوک های اختصاصی DSP	۲۵۸	۰	۰	۰	۱۰۰٪- (حذف مطلق)	استقلال کامل از منابع گران قیمت DSP

^{۷۵} Baseline Approx

^{۷۶} Intermediate Optimization

^{۷۷} Final Optimized Architecture

حافظه بلوکی (BRAM_۱۸K)	۱۰۱	۴۴	۴۴	۴۴	۵۶.۴٪ -	بهینه‌سازی دسترسی و حذف بافرهای واسط
جداول جستجو (LUT)	۴۵,۴۱۶	۱۶۱,۰۷۸	۱۳۴,۸۷۹	۹۸,۵۱۹	۱۱۶.۹٪ +	مهار مساحت (کاهش ۳۸.۸٪ نسبت به گام ۱)
فلیپ‌فلاپ‌ها (FF)	۵۸,۹۴۹	۷۰,۶۱۸	۶۹,۸۴۸	۶۶,۷۸۶	۱۳.۳٪ +	رشد جزئی ناشی از رجیسترهای خط لوله
درصد اشغال ناحیه SLR (LUT %)	۱۰٪	۳۷٪	۳۱٪	۲۲٪	-	قرارگیری در حاشیه کاملاً امن بدون احتقان
درصد اشغال کل تراشه (Total LUT %)	۲٪	۹٪	۷٪	۵٪	-	باقی ماندن ۹۵٪ ظرفیت تراشه برای توسعه
تأخیر مسیر بحرانی (Clock Period)	۴.۰۷۰ نانوثانیه	۳.۵۵۸ نانوثانیه	۳.۵۵۸ نانوثانیه	۳.۵۵۸ نانوثانیه	۱۲.۵٪ بهبود (سریع‌تر)	کوتاه‌تر شدن مدار ترکیبی و قابلیت افزایش کلاک
تأخیر پردازش برحسب سیکل (Latency)	۱۰۳۷ سیکل	۱۰۳۶ سیکل	۱۰۳۶ سیکل	۱۰۳۶ سیکل	۱ سیکل کاهش	حفظ کامل توان عملیاتی (Throughput)
زمان اجرای استنتاج (Absolute Time)	۵.۱۸۵ میکروثانیه	۵.۱۸۰ میکروثانیه	۵.۱۸۰ میکروثانیه	۵.۱۸۰ میکروثانیه	بدون جریمه زمانی	سرعت استنتاج پایدار و بلادرنگ

جدول ۳-۶: ارزیابی تطبیقی توان مصرفی و مشخصه‌های حرارتی استخراج شده از Vivado Power Report

دسته شاخص	پارامتر ارزیابی (Parameter)	یکا	مدل دقیق مبنا (Exact) (Baseline)	نسخه نهایی تقریبی (Proposed) (Approx)	میزان بهبود / تغییر	تحلیل مهندسی
توان مصرفی تفکیکی	توان دینامیک کل (Dynamic) (Power)	وات (W)	۰.۸۹۶	۰.۵۹۲	-۳۳.۹٪ (بهبود)	مهار شدید اتلاف سوئیچینگ
	توان استاتیک تراشه (Device) (Static)	وات (W)	۰.۱۳۴	۰.۱۳۳	-۰.۷٪	توان پایه نشتی سیلیکون
	توان سیگنال‌ها (Signals) (Power)	وات (W)	۰.۲۲۰	۰.۱۸۰	-۱۸.۲٪	کاهش بار خازنی خطوط داخلی
	توان مصرفی منطق (Logic) (Power)	وات (W)	۰.۰۹۹	۰.۱۶۲	+۶۳.۶٪	ناشی از جبران بار محاسباتی در LUTها
	توان بلوک‌های DSP	وات (W)	۰.۳۵۱ (۳۹٪)	۰.۰۰۰ (۰٪)	-۱۰۰٪ (حذف مطلق)	آزادسازی کامل ۳۵۱ میلی‌وات توان
	توان حافظه بلوکی (BRAM) (Power)	وات (W)	۰.۰۱۷	۰.۰۱۸	تغییر ناچیز	نرخ پایدار دسترسی به بافرها
	توان شبکه ساعت (Clocks) (Power)	وات (W)	۰.۱۲۰	۰.۱۴۴	+۲۰.۰٪	توزیع خطوط کلاک به LUTهای بیشتر

پورت‌های ورودی/خروجی ثابت	٪	۰.۰۸۸	۰.۰۸۸	وات (W)	توان ورودی/خروجی (I/O Power)	
سقوط توان به زیر ۱ وات	٪-۲۹.۴ (کاهش)	۰.۷۲۶	۱.۰۲۹	وات (W)	توان کل مصرفی Total On- (Chip Power)	توان کل
کارکرد در دمای بسیار خنک و پایدار	۰.۵ °C کاهش	۲۶.۴	۲۶.۹	سانتی‌گراد (°C)	دمای پیوند تراشه Junction Temp - T _j (°C)	مشخصه‌های حرارتی
فاصله بسیار امن تا آستانه حرارتی	۰.۵ °C افزایش	(۳۱.۰ ۵۸.۶ W ۳۱.۰ W)	(۵۸.۱ ۳۰.۷ W ۳۰.۷ W)	سانتی‌گراد (°C)	حاشیه اطمینان حرارتی Thermal Margin	
کاهش چشمگیر بار باتری در سامانه‌های Edge	٪-۲۹.۵ (بهبود)	۳.۷۶۰	۵.۳۳۵	میکروژول (μJ)	انرژی مصرفی هر استنتاج (Energy/Inf)	بهره‌وری انرژی
راندمان پردازشی فوق‌العاده به ازای هر وات	٪+۴۱.۹ (ارتقا)	۲۶۵,۹۳۷	۱۸۷,۳۹۷	فریم/وات	نرخ فریم بر توان (FPS / Watt)	

۶.۵. تحلیل جامع مهندسی و بازخوانی دستاوردهای سخت‌افزاری

تحلیل موشکافانه داده‌های سنتر در جدول فوق، چند واقعیت فنی کلیدی را آشکار می‌سازد:

مهار مصرف مساحت منطقی (LUT) در فرآیند بهینه‌سازی:

در فاز اولیه اعمال تقریب (گام ۱)، حذف بلوک‌های DSP با هزینه رشد مصرف جداول جستجو تا ۱۶۱,۰۷۸ همراه بود. اما با بازآرایی منطق و ادغام گیت‌ها در گام‌های بعدی، این عدد با ۳۸.۸٪ کاهش در نسخه نهایی به ۹۸,۵۱۹ رسید. این اصلاح مصرف مساحت را در سطح تنها ۲۲٪ از یک ناحیه SLR مهار کرد؛ امری که تضمین می‌کند فرآیند جاییابی و مسیریابی فیزیکی^{۷۸} بدون بروز هرگونه احتقان^{۷۹} در تراشه اجرا شود.

حذف کامل و ۱۰۰ درصدی بلوک‌های سخت‌افزاری DSP:

آزادسازی مطلق ۲۵۸ بلوک DSP و صفر کردن این شاخص، گلوگاه اصلی مقیاس‌پذیری مدار را رفع کرده است. این ویژگی دو مزیت استراتژیک به همراه دارد:

- امکان تکثیر و چندکاناله کردن موازی هسته‌های استنتاج روی تراشه بدون نگرانی از اشباع بلوک‌های DSP.

- قابلیت پورت کردن و اجرای روان معماری روی تراشه‌ها و پلتفرم‌های ارزان‌قیمت‌تر فاقد بلوک اختصاصی ضرب.

کاهش ۵۶.۴ درصدی مصرف حافظه بلوکی (BRAM):

کاهش مصرف ۱۸K BRAM از ۱۰۱ به ۴۴ بلوک مستقیماً ناشی از ساده‌سازی مسیر داده و حذف ساختار پیچیده ضرب دقیق است که نیاز به رجیسترهای بافرینگ طولی را در میان‌لایه‌ها از بین برده است.

بهبود ۱۲.۵ درصدی زمان‌بندی مسیر بحرانی:

کاهش تأخیر بحرانی از ۴۰.۷۰ به ۳.۵۵۸ نانوثانیه نشان می‌دهد که منطق ترکیبی سبک‌تر شده و مدار بر روی کلاک هدف ۵ نانوثانیه (۲۰۰ مگاهرتز) از یک حاشیه اطمینان زمانی^{۸۰} معادل ۱.۳۵ نانوثانیه برخوردار است که توانایی کار در فرکانس‌های بالاتر تا ۲۸۰ مگاهرتز را به اثبات می‌رساند.

۶.۶. تحلیل رفتار توان مصرفی، پایداری حرارتی و بهره‌وری انرژی

^{۷۸} Placement & Routing

^{۷۹} Congestion

^{۸۰} Slack

بررسی نتایج جدول ۳-۶ که از ابزار تحلیل توان Vivado بر مبنای بردار فعالیت شبکه‌های سنتز شده استخراج شده است، برتری قاطع معماری تقریبی را در زمینه‌ی بهره‌وری انرژی به اثبات می‌رساند:

حذف کامل ۳۵۱ میلی‌وات توان دینامیک DSP:

در مدل دقیق، بلوک‌های DSP با مصرف ۰.۳۵۱ وات، به‌تنهایی ۳۹٪ از کل توان دینامیک تراشه را به خود اختصاص داده بودند. با جایگزینی جدید سهم توان DSP به صفر مطلق رسیده است. اگرچه توان بخش منطق (Logic) به دلیل رشد تعداد LUTها از ۰.۰۹۹ وات به ۰.۱۶۲ وات افزایش یافته، اما صرفه‌جویی ناشی از حذف DSP چنان بزرگ بوده است که در مجموع توان دینامیک با کاهش ۳۳.۹ درصدی از ۰.۸۹۶ وات به ۰.۵۹۲ وات سقوط کرده است.

شکستن مرز توان ۱ وات در سطح کل تراشه:

توان کل تراشه^{۸۱} با ۲۹.۴٪ کاهش از ۱.۰۲۹ وات به ۰.۷۲۶ وات رسیده است. این کاهش توان ۲۹.۴ درصدی، شتاب‌دهنده را از رده پردازنده‌های پرمصرف خارج کرده و امکان استقرار مستقیم آن را بر روی پلتفرم‌های لبه و سامانه‌های تغذیه‌شونده با باتری یا منابع محدود بدون نیاز به مدارهای پیچیده مدیریت تغذیه فراهم می‌سازد.

کاهش دمای پیوند (T_j) و افزایش حاشیه امن حرارتی:

به دلیل کاهش اتلاف توان در بستر سیلیکون، دمای پیوند تراشه در دمای محیط ۲۵ درجه سانتی‌گراد از 26.9°C به 26.4°C افت کرده و حاشیه اطمینان حرارتی^{۸۲} به 58.6°C (معادل تحمل ۳۱.۰ وات توان حرارتی اضافه) افزایش یافته است. این پایداری حرارتی نیاز به فن یا هیت‌سینک‌های حجیم را منتفی می‌کند.

بهبود ۲۹.۵ درصدی شاخص انرژی و جهش ۴۱.۹ درصدی راندمان:

با ضرب توان کل در زمان تاخیر استنتاج، انرژی مصرفی به ازای هر طبقه‌بندی تصویر از ۵.۳۳۵ میکروژول به ۳.۷۶۰ میکروژول کاهش یافته است (۲۹.۵٪ صرفه‌جویی خالص در انرژی). همچنین شاخص راندمان عملیاتی به ازای توان مصرفی (FPS/Watt) از ۱۸۷,۳۹۷ به ۲۶۵,۹۳۷ فریم بر ثانیه به ازای هر وات جهش یافته است (۴۱.۹٪ ارتقای بازدهی پردازش).

^{۸۱} Total On-Chip Power

^{۸۲} Thermal Margin

۷. فصل هفتم: جمع‌بندی و کارهای آینده

۷.۱. جمع‌بندی یافته‌ها

هدف از این پژوهش، گذار از پارادایم سنتی «تکیه بر بلوک‌های DSP» در شتاب‌دهنده‌های هوش مصنوعی به سمت منطق انعطاف‌پذیر و بهینه‌سازی منابع سخت‌افزاری بود.

در فازهای نخستین این مطالعه، ساختارهای ضرب‌کننده تقریبی مبتنی بر فشرده‌سازهای ۴ به ۲ (۴:۲ Compressors) مورد ارزیابی قرار گرفتند و چند نمونه از آن‌ها پیاده‌سازی شدند. با این حال، تحلیل‌های فنی نشان داد که پیاده‌سازی فشرده‌سازهای سطح گیت در بستر تراشه‌های به دلیل عدم انطباق بهینه با ساختار داخلی LUT-۶ و مسیرهای بحرانی سنتز سطح بالا (HLS)، سربار مسیریابی و تاخیر قابل‌توجهی به همراه دارد. از این رو، استراتژی طراحی به سمت کوانتیزه‌سازی توان دو (Power-of-Two یا PoT) و جایگزینی ضرب عمومی با عملیات شیفت بیتی هدایت شد. این رویکرد ثابت کرد که در شبکه‌های عصبی با پیچیدگی متوسط (مانند CNN مورد ارزیابی)، منابع DSP اختصاصی کاملاً قابل‌حذف هستند و شیفت‌دهنده‌های مبتنی بر منطق بیتی، بازدهی سخت‌افزاری و توان مصرفی به مراتب بهتری نسبت به ضرب‌کننده‌های تقریبی کلاسیک ارائه می‌دهند.

۷.۲. ارزیابی تطبیقی در بستر آکادمیک

روش‌های ارائه‌شده در کارهای کلاسیک عمدتاً بر اصلاح سطح گیت مدارات حسابی و استفاده از ساختارهای فشرده‌ساز تقریبی تمرکز دارند. با وجود برتری این مدارات در سنتز ASIC، پیاده‌سازی آن‌ها در FPGAهای مدرن به‌علت تحمیل تأخیرات Routing در میان اسلایس‌ها و تداخل با منطق پیش‌ساخته‌ی تراشه، اغلب مزیت توان و فرکانس را خنثی می‌کند؛ واقعیتی که در ارزیابی‌های اولیه این پژوهش نیز مشخص گردید. در نقطه مقابل، رویکرد PoT پیاده‌سازی‌شده به ما نشان داد که در چارچوب‌های HLS-based نظیر hls4ml حذف ریاضی ضرب‌کننده و اتکا به شیفت، همگرایی بسیار بهتری با ابزارهای سنتز خودکار دارد و به شکلی کارآمدتر از رویکردهای سخت‌افزاری سطح مدار، فضا و مصرف توان را کاهش می‌دهد.

۷.۳. محدودیت‌های ذاتی پژوهش

طراحی و پیاده‌سازی معماری پیشنهادی در این پژوهش با چند چالش و محدودیت کلیدی مواجه بود که در تحلیل نتایج باید مد نظر قرار گیرند:

افت دقت ناشی از کوانتیزه‌سازی غیریکواخت پس از آموزش (Post-Training Quantization Gap):

انتقال وزن‌های آموزش‌دیده در فاز شناور به فضای توان دو به‌صورت Post-Training انجام شد. از آنجا که فواصل اعداد در نمایش PoT به‌صورت توان‌های نمایی افزایش می‌یابند، کوانتیزه‌سازی وزن‌های بزرگ‌تر با خطای مطلق بیشتری همراه است. عدم بازآموزی (Retraining) یا عدم استفاده از الگوریتم‌های آگاه از تقریب در حین آموزش، بخشی از افت ۲/۱ درصدی دقت را به شبکه تحمیل کرد.

سربار منطق کنترل و مسیریابی در ابزار HLS:

با اینکه استفاده از توابع شیفت منجر به حذف کامل DSP‌ها شد، اما مدیریت شیفت‌های پویا و بررسی شرط صفر بودن وزن‌ها (nonzero) در سطح HLS، باعث افزایش مصرف منابع منطقی (LUT‌ها و مالتی‌پلیرها) گردید. ابزارهای HLS به دلیل نگاشت محافظه‌کارانه منطق بیتی به سلول‌های ۶-LUT در FPGA، گاهی کدهای ++C را به مدارات منطقی بزرگ‌تر از حد بهینه متناظر در RTL تبدیل می‌کنند.

وابستگی به ابزار و معماری تراشه (Platform Dependency):

نتایج به دست آمده از سنتز توان و منابع، وابستگی شدیدی به ساختار فیزیکی Virtex UltraScale+ (تراشه xcuv250) دارد. نحوه توزیع اسلایس‌ها و اتصال داخلی LUT‌ها در این معماری، رفتار تأخیر و توان را به شدت تحت تأثیر قرار می‌دهد و تعمیم مستقیم ارقام مصرف توان به تراشه‌های کم‌مصرف‌تر (مانند خانواده Zynq یا Artix) نیازمند سنتز مجدد در آن بسترها بود.

۷.۴. پیشنهادات برای پژوهش‌های آتی

جهت توسعه و ادامه این مسیر، موارد زیر پیشنهاد می‌گردد:

۱. آموزش آگاه از تقریب^{۸۳}؛ از آنجا که کوانتیزه‌سازی به توان دو در این کار به‌صورت پس از آموزش (Post-Training) اعمال شد، گنجاندن این قید در حلقه یادگیری مدل می‌تواند افت دقت حاصله را به حداقل برساند.
۲. بررسی اثرات فشرده‌سازها در مقیاس هیبریدی: ارزیابی ترکیبی فشرده‌سازهای ۴ به ۲ فقط در لایه‌های انتهایی یا باینری در کنار شیفت PoT در لایه‌های کانولوشن، می‌تواند به‌عنوان یک ساختار ناهمگن بررسی شود.
۳. پویاسازی نرخ تقریب^{۸۴}: بهره‌گیری از تقریب انتخابی، به‌گونه‌ای که لایه‌های حساس شبکه با دقت کامل و لایه‌های با مقاومت بالاتر نسبت به نویز، به فرمت PoT کوانتیزه گردند.
۴. توسعه بومی در اکوسیستم hls4ml: طراحی ماژول اختصاصی و استاندارد درون hls4ml جهت اتوماسیون کامل تبدیل وزن‌ها به فرمت توان دو و تزریق توابع شیفت بدون نیاز به پیچ‌کردن دستی فایل‌های هسته.
۵. ارزیابی در مدل‌های عمیق‌تر: سنجش مقیاس‌پذیری این روش در شبکه‌های پیچیده‌تر نظیر ResNet بر روی دیتاست‌های سنگین‌تر برای تحلیل اثر تجمعی خطای تقریب در زنجیره طولانی لایه‌ها.

^{۸۳} Approximate-Aware Training

^{۸۴} Adaptive Precision

منابع و پیوست

[1]A. M. Dalloo, A. J. Humaidi, A. K. Al-Mhdawi, and H. Al-Raweshidy, "Approximate Computing: Concepts, Architectures, Challenges, Applications, and Future Directions," IEEE Access, vol. 11, pp. 49303–49326, 2023.

[2]H. Jiang, F. J. H. Santiago, H. Mo, L. Liu, and J. Han, "Approximate Arithmetic Circuits: A Survey, Characterization, and Recent Applications," Proceedings of the IEEE, vol. 108, no. 12, pp. 2108–2135, Dec. 2020.

[3]Y. Wu et al., "A Survey on Approximate Multiplier Designs for Energy Efficiency: From Algorithms to Circuits," ACM Computing Surveys, vol. 56, no. 1, pp. 1–37, 2023.

[4]S. Shakibhamedan, N. Amirafshar, A. S. Baroughi, H. S. Shahhoseini, and N. Taherinejad, "ACE-CNN: Approximate Carry Disregard Multipliers for Energy-Efficient CNN-Based Image Classification," IEEE Transactions on Emerging Topics in Computing, vol. 12, no. 1, pp. 15–28, Jan.-March 2024.

[5]S. Hwang, H. Seok, and Y. Kim, "Design of an Approximate 4-2 Compressor with Error Recovery for Efficient Approximate Multiplication," IEEE Access, vol. 11, pp. 78201–78210, 2023.

[6] A. Momeni, J. Han, P. Montuschi, and F. Lombardi, "Design and Evaluation of Approximate 4-2 Compressors for High-Speed and Energy-Efficient Approximate Multipliers," IEEE Transactions on Computers, vol. 64, no. 4, pp. 1040–1053, Apr. 2015.

[7] O. Akbari, M. Kamal, A. Afzali-Kusha, and M. Pedram, "Dual-Quality 4:2 Compressors for Dual-Quality Approximate Multipliers," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 25, no. 1, pp. 335–339, Jan. 2017.

[8] F. Fahim et al., "hls4ml: An open-source codesign workflow to deploy deep neural networks on FPGAs," *Machine Learning: Science and Technology*, vol. 2, no. 4, Art. no. 045015, 2021.

[9] S. Zhou, Y. Wu, Z. Ni, X. Zhou, H. Wen, and Y. Zou, "DoReFa-Net: Training Low Bitwidth Convolutional Neural Networks with Low Bitwidth Gradients," *arXiv preprint arXiv:1606.06160*, 2016.

[10] V. Mrazek, Z. Vasicek, L. Sekanina, H. Jiang, and J. Han, "Design of Approximate Multipliers Using Genetic Programming," *IEEE Transactions on Emerging Topics in Computing*, vol. 7, no. 4, pp. 605–617, Oct.-Dec. 2019.

[11] M. Blott et al., "FINN-R: An End-to-End Deep-Learning Accelerator Generation Framework for Extreme Throughput at the Edge," *ACM Transactions on Reconfigurable Technology and Systems*, vol. 14, no. 3, pp. 1–28, 2021.

[12] Y. Umuroglu et al., "FINN: A Framework for Fast, Scalable Binarized Neural Network Inference," in *Proceedings of the 2017 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA '17)*, Monterey, CA, USA, 2017, pp. 65–74.